



Actionable Knowledge With Context Graphs



William Lyon
Product @ Neo4j
lyonwj.com
[@lyonwj](https://twitter.com/lyonwj)



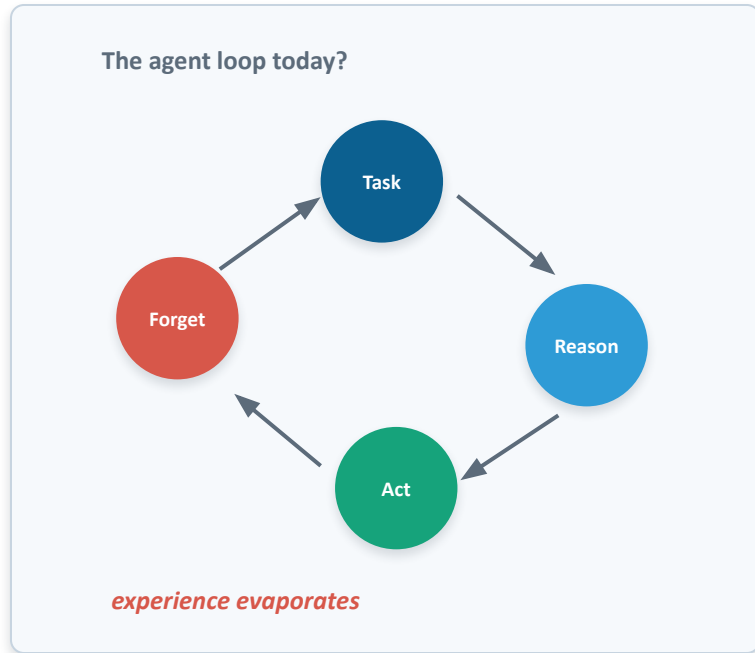
Actionable Knowledge with Context Graphs

Turning agent memory into provenance-grounded, executable skills.

Your agent is brilliant — and amnesiac

Every run, your agent re-derives behavior from scratch. It solved this exact incident last week — and remembers nothing. The bottleneck for agents in production isn't reasoning. It's turning experience into reusable, trustworthy knowledge.

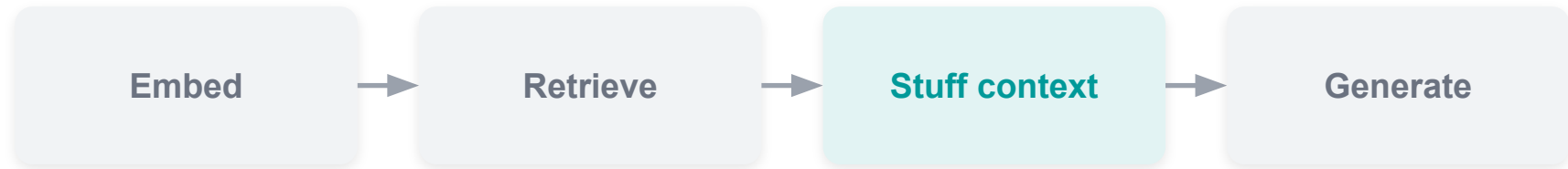
- **Context windows are short-term memory** — great for a turn, useless across sessions
- **Fine-tuning is slow and opaque** — you can't audit or revert what it 'learned'
- **Prompts and RAG recall text** — not procedures the agent can act on



THE PROBLEM

The pattern under “agent memory”

Embed everything. Retrieve what looks similar. Stuff it into the context window. Hope the model figures out what to do — **on every turn.**

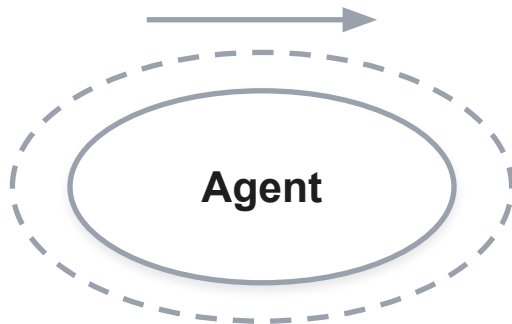


A retrieval loop. Useful — but it is a lookup, repeated.

THE PROMISE

Your agents have memory now

Retrieval-augmented, vector-backed, context-stuffed. So why do they still feel like they start over on every turn?



re-derive what to do, every turn

“Memory” has mostly meant remembering text — not knowing what to do with it.

Three failures between experience and reuse

1

Forgetting

No durable, queryable store of what happened. Each session starts cold; nothing persists or compounds across runs.

2

No transfer

Even with logs, raw transcripts don't generalize. The agent can't lift a reusable procedure out of last week's success.

3

No provenance

If an agent 'learns' a rule, can you trust it? Ungrounded, un-auditable knowledge is a liability, not an asset.

Context graphs attack all three at once — durable store, graph structure for transfer, edges for provenance.

THE PROBLEM

What the agent actually receives

A pile of relevant text — not a model of who, what, or why.



THE PROBLEM

Genuinely useful — and genuinely limited

Recall is good at

- surfacing relevant passages
- fuzzy lookup over long history
- cheap, fast, general

But it can't

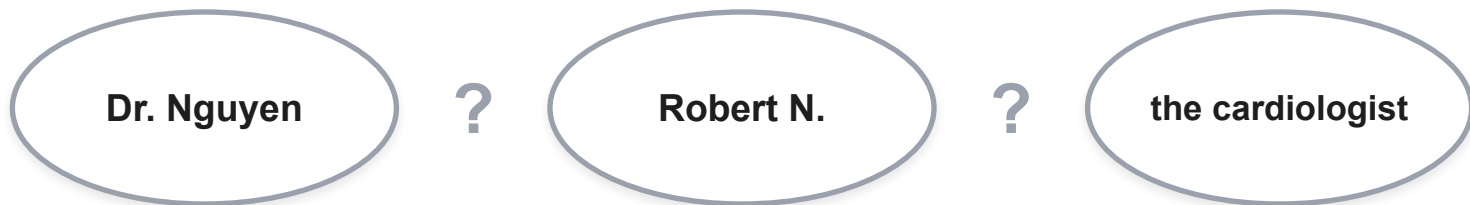
- resolve that two mentions are one entity
- say why a past decision was made
- hand structured know-how to the next agent

So the model rebuilds meaning from scratch — every single turn.

THE PROBLEM

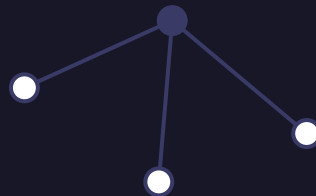
Recall isn't knowledge an agent can act on

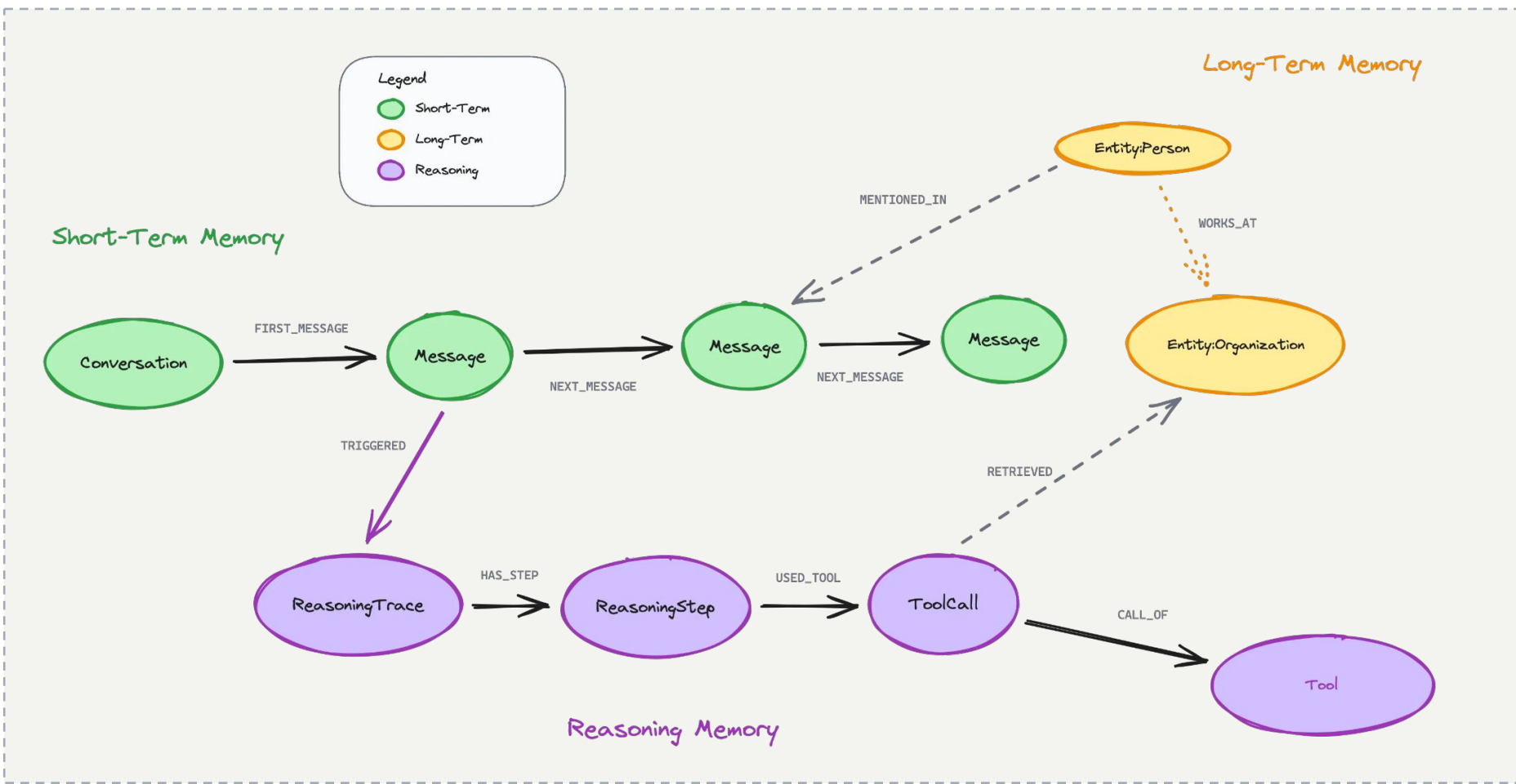
Retrieve every mention of one customer and the agent still has three disconnected strings — not one person.

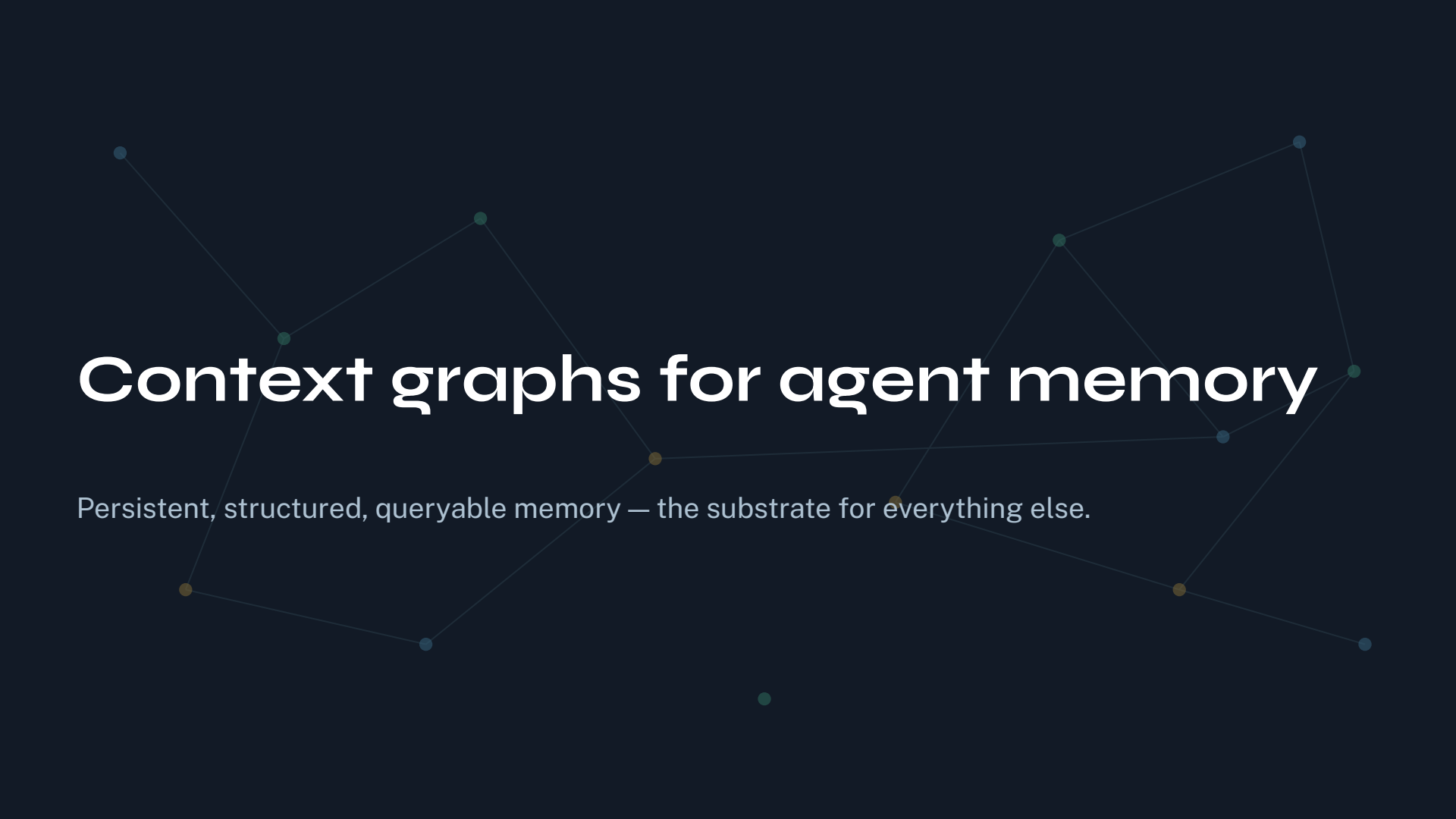


Same person. Three surface forms. Zero connection — and often a confidently invented fourth.

To act, an agent needs knowledge that's **connected, typed, and traceable** — not just retrievable.







Context graphs for agent memory

Persistent, structured, queryable memory — the substrate for everything else.

Why not just vectors?

Vector / RAG memory

- **Recall by similarity** — nearest-neighbor over flat chunks
- **No structure** — chunks don't know how they relate
- **No identity resolution** — 'Acme Corp' $\neq$ 'Acme' across docs
- **Hard to audit** — why was this chunk retrieved?

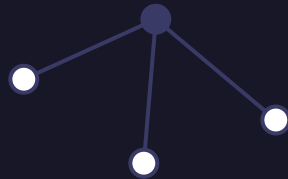
Context graph memory

- **Entities + relationships** — a typed, traversable model of the world
- **Resolution & consolidation** — one node per real-world thing
- **Provenance by construction** — edges link claims to sources
- **Hybrid, not either/or** — HNSW vector indexes live ON the graph

The point isn't to replace vectors — it's to give them a skeleton.

From recall to action — and a live demo

- 1 Why recall isn't enough
- 2 The context graph — and three memory types in one graph
- 3 What agents know, how they decided, how to act
- 4 Agents that share memory — and learn from each other
- 5 Skills that run, not prose to re-interpret
- ▶ Demo: two agents, one graph



Most agent memory is just recall

Necessary — but not the same as knowing what to do.

A graph-native picture of what your agents know, how they came to know it, and how to act on it.



What they know

Resolved, typed entities and relationships



How they learned it

Decisions and provenance as structure



How to act

Skills as runnable steps, not prose

Three memory types, one knowledge graph

Short-term

:Message

conversation, sessions,
recent history

Long-term

POLE+O entities

people, orgs, places,
events, objects — linked

Reasoning

:Decision

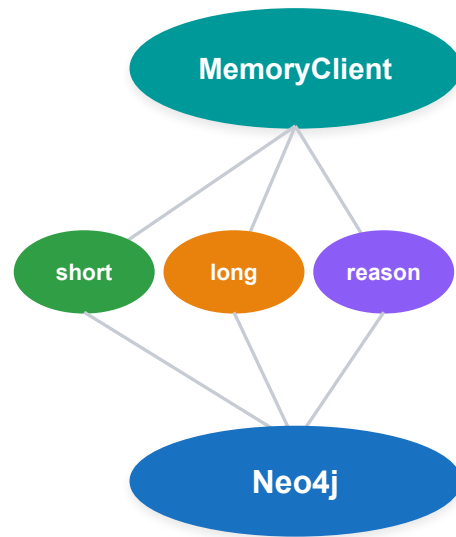
decision traces &
tool-call provenance

Reasoning is the differentiator — most memory systems stop at the first two.

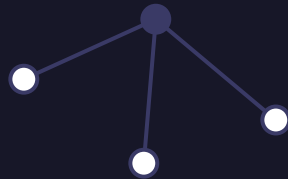
One client, three layers, one graph

```
from neo4j_agent_memory import MemoryClient

async with MemoryClient(settings) as memory:
    await memory.short_term.add_message(sid, role, text)
    await memory.long_term.add_entity("Dr. Nguyen", "PERSON")
    ctx = await memory.get_context(query, sid)
```



The context graph lives in the same Neo4j as your domain data — not a bolted-on store.



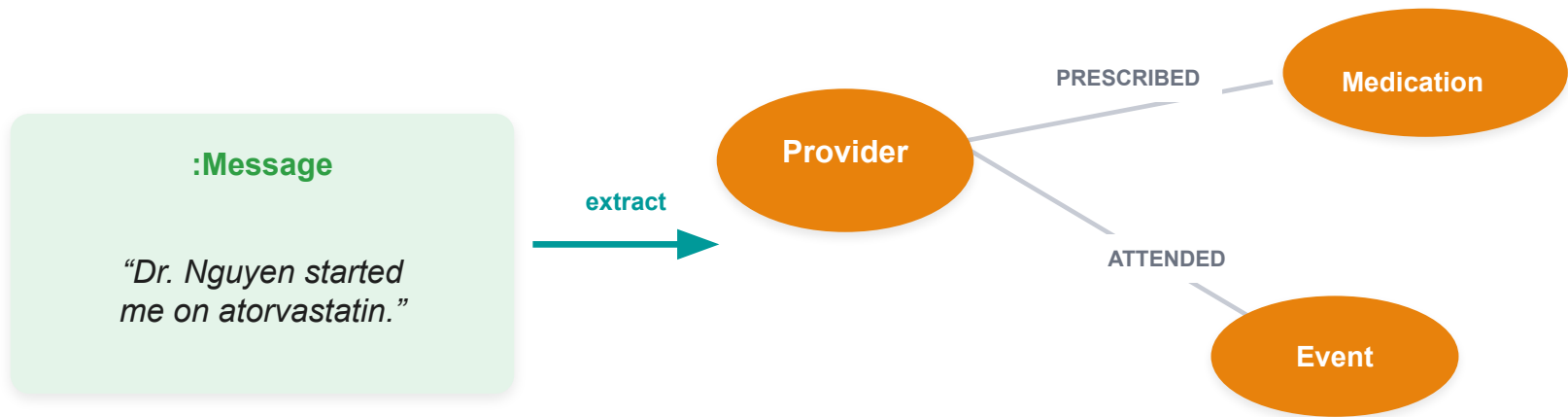
Typed, resolved entities

Long-term memory, built from conversation.

WHAT THEY KNOW

From conversation to graph

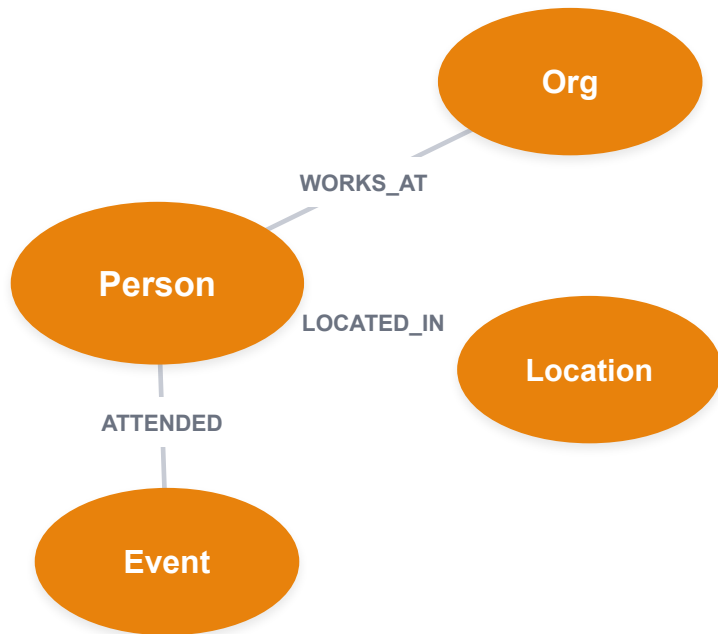
Messages land in short-term memory; extraction builds the long-term graph behind them
— no manual schema wrangling.



WHAT THEY KNOW

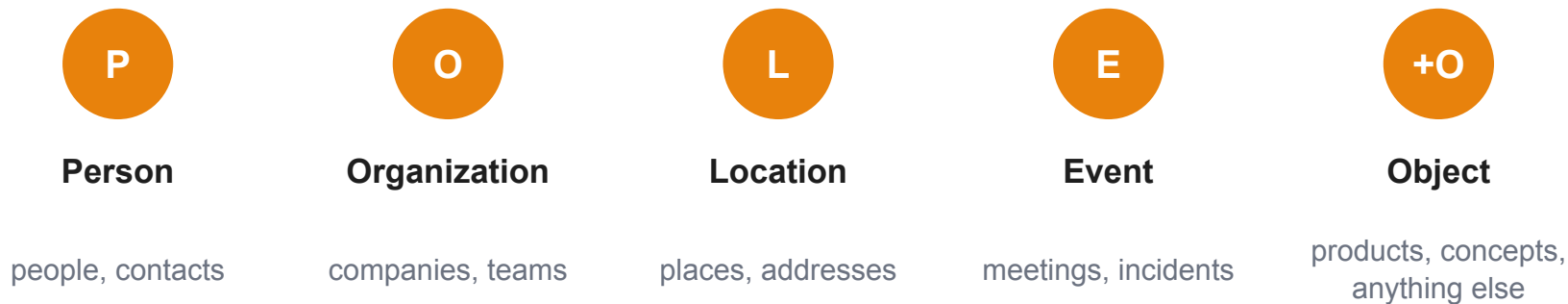
Typed entities, typed relationships

Every node has a type; every edge has a type.
The agent **traverses to exactly what bears on a decision** — instead of ranking nearest neighbours and guessing.



The POLE+O model

Long-term memory classifies entities into five types — the canonical schema for the context graph.

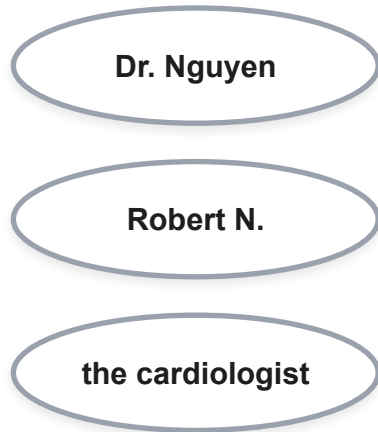


Resolved and deduplicated, so the same entity is one node — however it's phrased.

WHAT THEY KNOW

Entity resolution: three forms, one node

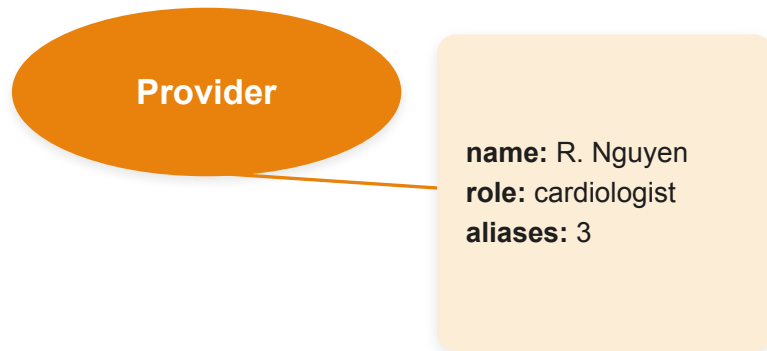
BEFORE



resolve



AFTER

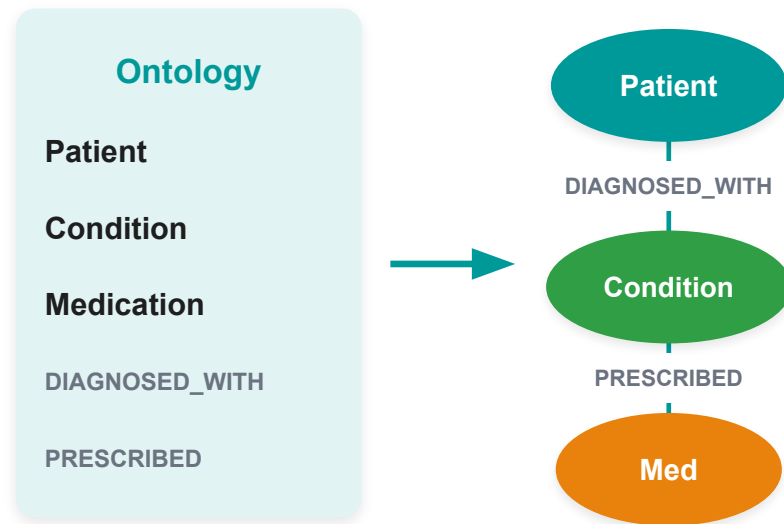


Facts attach to the entity — not scattered across passages.

A shared ontology

Define the types that matter for your domain;
extraction produces **typed, validated,
deduplicated** entities — not generic blobs.

Same conversation, shaped to your world.

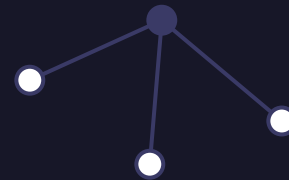


Ask the graph a precise question

```
MATCH (p:Person {name: "Dr. Nguyen"})-[:PRESCRIBED]->(m:Object)
MATCH (p)-[:WORKS_AT]->(o:Organization)
RETURN m.name, o.name
```

One hop to the medication, one to the practice — grounded in your data, not a guess from the nearest paragraph.

HOW THEY DECIDED



Reasoning as first-class structure

HOW THEY DECIDED

Store the thinking, not just the facts

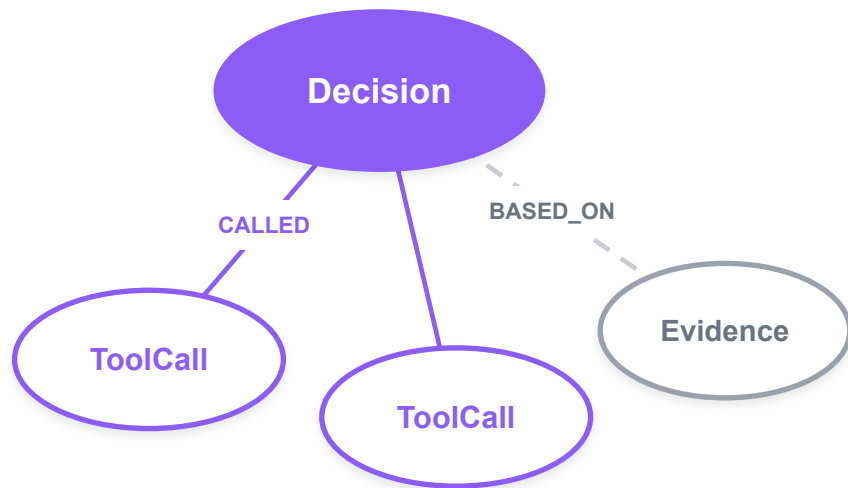
Most systems remember **what** an agent learned. Few remember **how it decided**. The context graph stores decisions, tool calls, and outcomes as nodes.

thought → **action** → **outcome**, captured and queryable.

So an agent can learn from its own past reasoning — and, soon, from other agents’.

A decision trace

Each decision links to the tools it called and the evidence it rested on — full provenance.

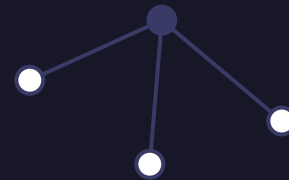


Not a log line — a connected subgraph.

See why — not just what

```
MATCH (d:Decision)-[:BASED_ON]->(e)
WHERE d.outcome = "escalated"
RETURN d.summary, collect(e)
```

Ask the graph why the agent escalated — and get the evidence it used, every time.

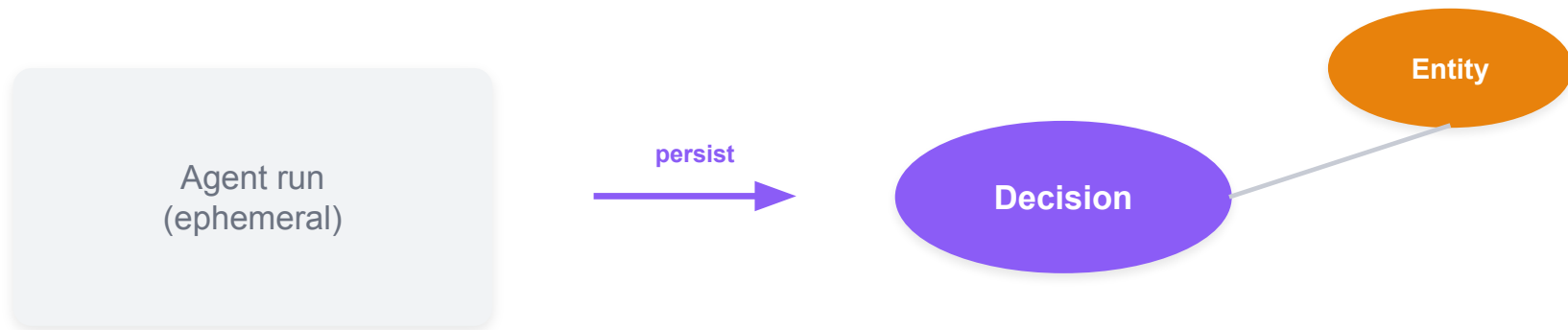


Agents learn from each other

One agent's reasoning becomes another's context.

A trace isn't trapped in one run

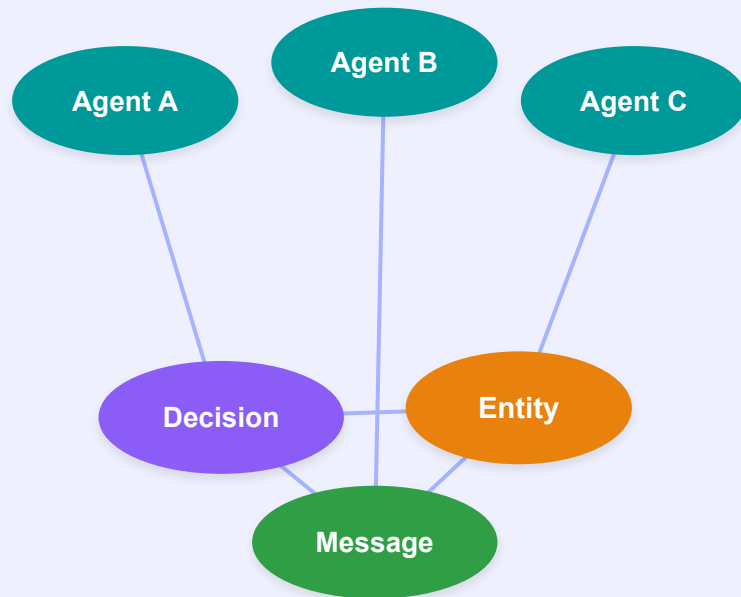
Because reasoning is a **graph object**, not a transcript, it outlives the session that created it — and any agent with access can read it.



Persisted once. Available to everyone after.

One shared context graph

Many agents, one graph. Each writes what it learns and decides; each reads what the others already built.



one shared graph · many agents

Agent A writes, Agent B reads



Agent B starts from Agent A's resolved entities and decisions — not a cold context window.

This is the demo you're about to see.

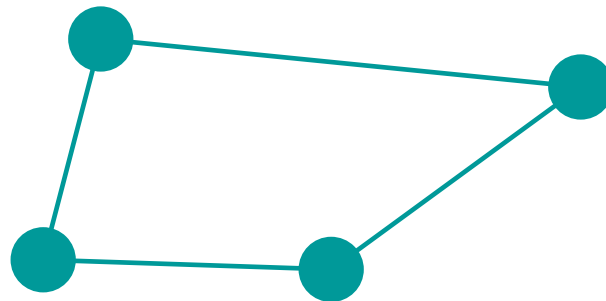
THE IDEA

Actionable knowledge

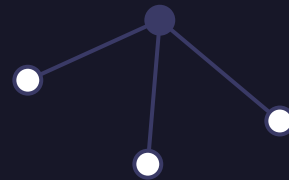
Turn what your agents store into knowledge **structured so they can act on it** — not text they must re-interpret first.



recall



context graph



Extraction & a shared ontology

What goes in the graph — and in what shape.

How entities get built

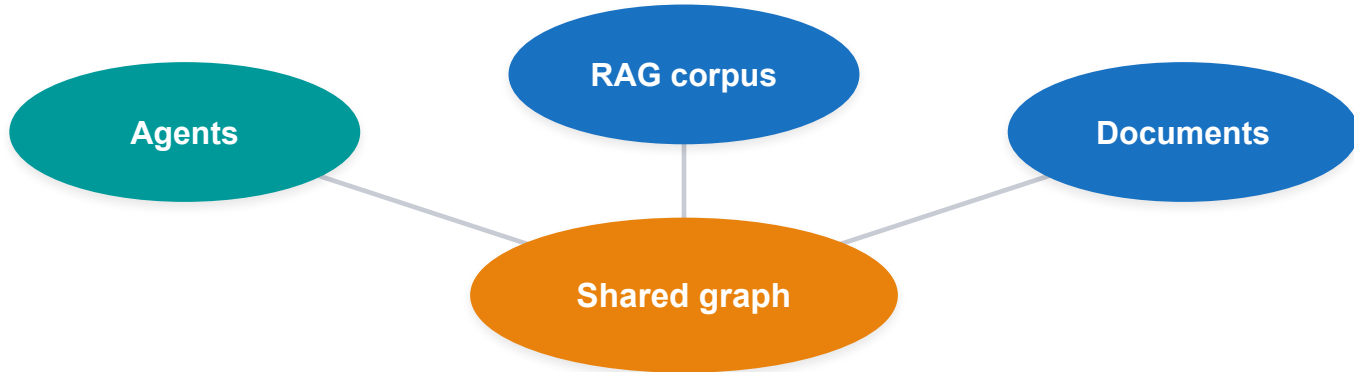
A staged pipeline trades cost for accuracy — fast statistical NER first, zero-shot typing next, an LLM only when needed.

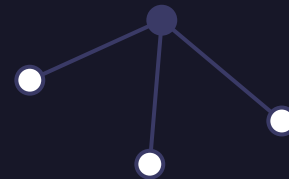


Enrichment (Wikipedia, Diffbot) hydrates entities with context in the background.

One shape for agents and source data

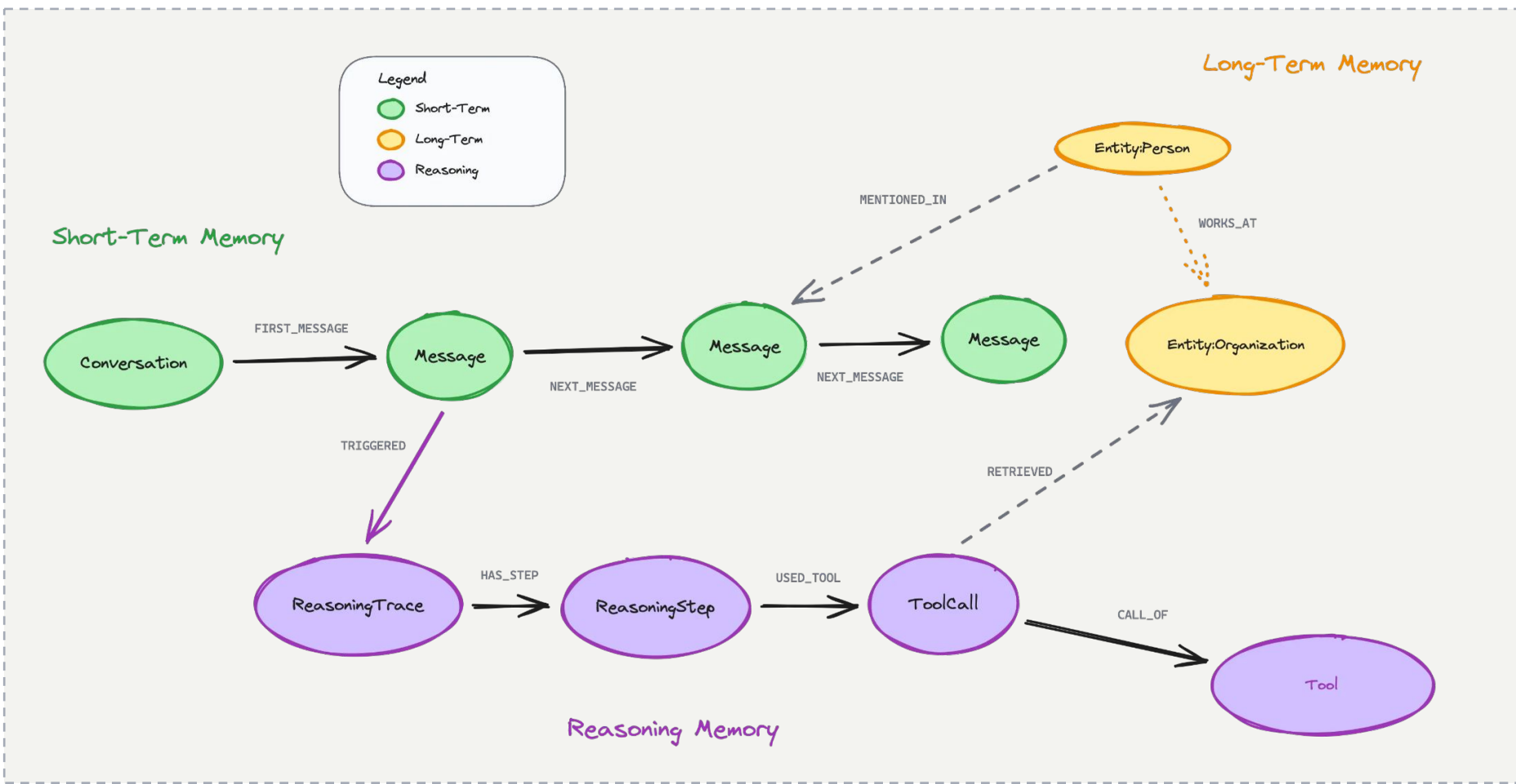
The same ontology governs what every agent extracts and how your documents and RAG corpus are ingested — so it all lands in one consistent graph.





From recall to actionable knowledge

Knowledge structured so an agent can act on it.



Memory tells you **what happened**.
Skills tell you **what to do**.



Distillation is the compiler from short-term, long-term, and reasoning memory into governed, reusable procedural knowledge.

Two kinds of knowledge

Descriptive

Tells you what is — facts retrieved, context recalled. Inert until something interprets it. This is what recall gives you.

Actionable

Structured so an agent can act — resolved entities, traceable reasoning, runnable steps. This is what a graph gives you.

A skill: SKILL.md + progressive disclosure

1 · Metadata

name + description — always in context; the trigger

2 · SKILL.md body

loaded when relevant — the core procedure

3 · References & code

pulled on demand — exemplars, schema, scripts

agentskills.io — an open standard

A folder + a Markdown file with YAML frontmatter. Portable across Claude, Code, the Agent SDK — and any compatible runtime.

Anthropic, Oct 2025

“We hope to enable agents to create, edit, and evaluate Skills on their own, letting them codify their own patterns of behavior into reusable capabilities.”

Progressive disclosure → effectively unbounded context, loaded only as needed.

Prose tells; a graph runs

A prose skill

“Validate the patient, check interactions, then recommend.” Re-read and re-interpreted on every run.

A skill as a graph

Typed steps with typed inputs and outputs. Vetted once, run reliably, debugged step by step.

Codify effective actions so they repeat — instead of reconstructing them each time.

AIP: A Graph Representation for Learning and Governing Agent Skills

Zach Blumenfeld
Neo4j
USA

Jim Webber
Neo4j
UK

AIP: A Graph Representation for Learning and Governing Agent Skills

Before: human-authored prose (SKILL.md)

```
# Exoplanet Detection Workflows
## Pipeline Design Principles
### Key Stages
1. Data Loading: format, columns, time system
2. Quality Control: filter via quality flags
3. Preprocessing: remove noise, preserve signal
4. Period Search: choose algorithm for signal
5. Validation: verify candidate is real
6. Refinement: improve period precision

### Critical Decisions
Which period search algorithm?
- TLS: best for transit-shaped dips (box-like)
- Lomb-Scargle: any periodic signal, fast
- BLS: alternative to TLS, in Astropy
```

After: compiled AIP procedure (SKILL.md, YAML)

```
purpose: >
  Load, quality-filter, preprocess, search for a transit
  signal, validate vs SDE/SNR thresholds, refine period.
steps:
  - name: load-and-inspect
    outputs:
      - (name: lc-path, type: string)
      - (name: column-map, type: object)
      - (name: flag-good-value, type: float)
    - name: choose-search-strategy # judgment node
      description: >
        Default to TLS; load references/method-selection.md
        to justify TLS vs Lomb-Scargle vs BLS.
        one_of: [tls, lomb-scargle, bls]
      outputs: [(name: method, type: string)]
    - name: run-detection-pipeline # script node
      script: scripts/detect_period.py
      inputs: [lc-path, column-map, flag-good-value]
      outputs: [(name: detection, type: object)]
    - name: validate-candidate # script node
      script: scripts/validate_candidate.py
      inputs: [detection]
      outputs: [(name: verdict, type: object)]
```

On-disk skill package (before → after)

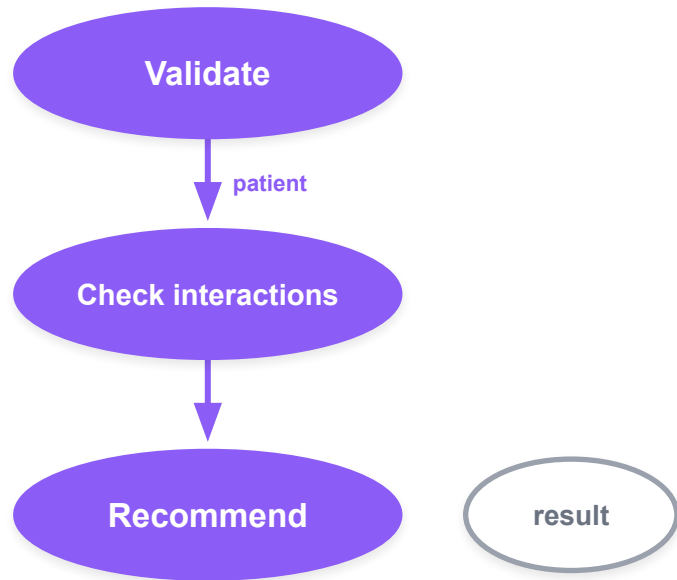
```
# Before: human-curated (prose-only)
exoplanet-workflows/
  SKILL.md # prose guidance only
```

```
# After: compiled AIP (exoplanet-workflows)
exoplanet-workflows/
  SKILL.md # AIP procedure (YAML above)
  scripts/ # deterministic step bodies
    detect_period.py
    period_range_guide.py
    validate_candidate.py
  references/ # prose for judgment steps
    method-selection.md
    troubleshooting.md
  source/ # provenance kept by compiler
    original-SKILL.md
    procedure.schema.json
```

Figure 3: Compiling a prose skill to AIP, for exoplanet-workflows. The free-form procedure (top left) becomes a schema-validated YAML graph of typed steps (top right): each step declares typed inputs/outputs, binds to a deterministic script via script, or cites a prose reference from its description for judgment. On disk (bottom), the prose-only human skill—exoplanet-workflows, one of five modules the task bundles, all shipping no code—expands into a package whose steps are backed by generated scripts/ and references/, with the original prose and JSON schema preserved under source/; Figure 2 is this same skill as a graph.

Skills as typed execution graphs

Compile a skill from prose into **steps as nodes**,
typed I/O edges between them — projected into
the same graph as memory and reasoning.



Why prose skills break — and what AIP proposes

Prose instructions carry four compounding problems. AIP represents a skill as a typed, schema-validated DAG — a validity-gated action space that measurably beats prose.

Prose skills break

a Markdown file of prose — four compounding problems

✗ Re-derivation cost

Re-read and re-interpreted every session — slow, token-heavy, with run-to-run variance.

✗ Prompt brittleness

Semantics-preserving wording changes can swing task accuracy by tens of points.

✗ Resistance to improvement

Additive bias — agents pile on words rather than fix structure they don't fully grasp.

✗ Opaque failure

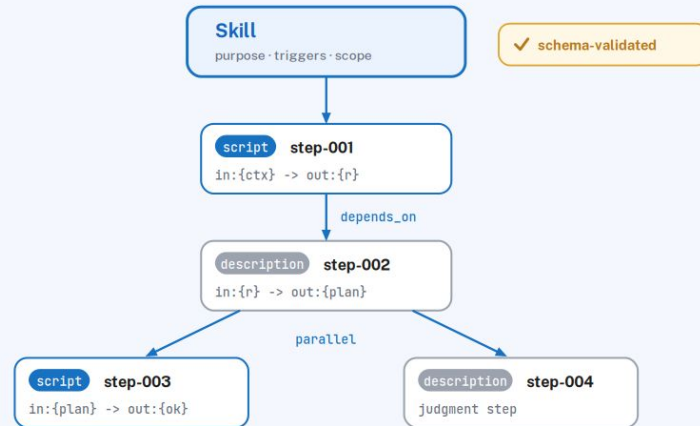
No addressable units, so a failure can't be attributed to a specific step.

AIP

typed · schema-validated

AIP: a typed, schema-validated DAG

a skill node over typed step nodes, validated at authoring time

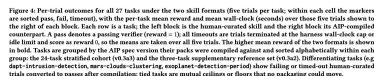


control edges: `depends_on` · `parallel` · `one_of` · typed I/O between steps

Validity-gated action space — every node typed, schema-bound and addressable for diagnosis and repair.

Handed runnable graph steps instead of prose to reinterpret, the agent acts more reliably — and is far easier to debug.

Handed runnable graph steps instead of prose to reinterpret, the agent acts more reliably — and is far easier to debug.



0.60 $\rightarrow$ **0.71**

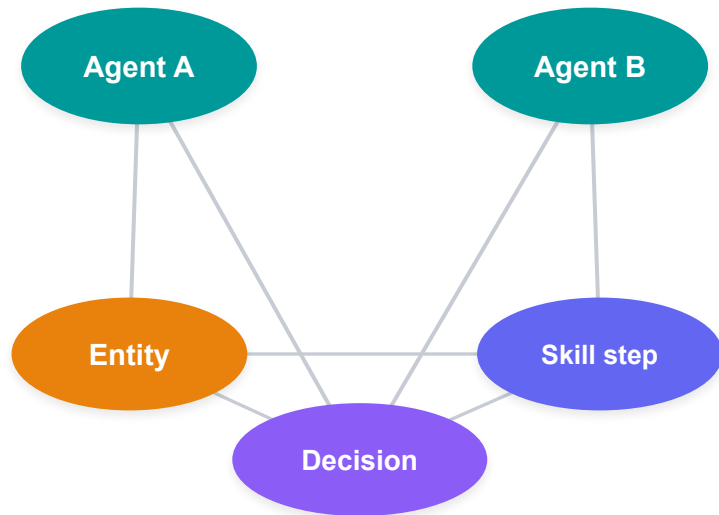
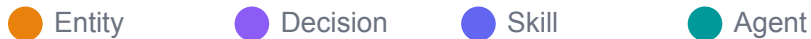
53% → 67%

Same skill, compiled to a graph vs. left as prose · SkillsBench · AIP (Blumenfeld & Webber, 2026)

PUTTING IT TOGETHER

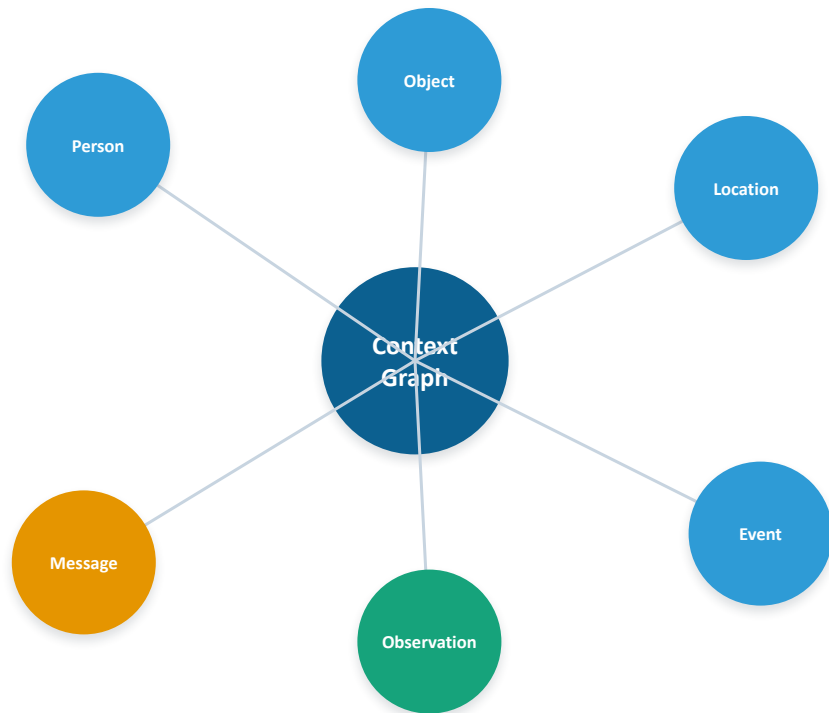
One substrate

Knowledge, reasoning, skills — **and the agents themselves** — in one queryable graph, in the same Neo4j as your domain data.



One place to query — and exactly what we'll watch two agents build, live, next.

A context graph: POLE+O, episodes, and reasoning



Maps to the cognitive memory model (CoALA)



Episodic

Messages — raw turns, what literally happened



Semantic

POLE+O entities & observations — abstracted facts

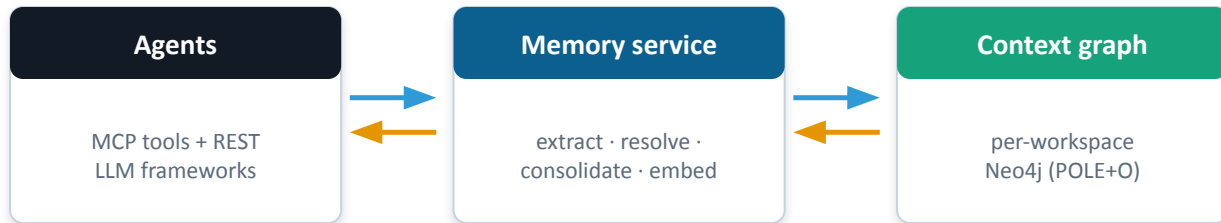


Procedural

AgentStep + ToolCall traces — how the agent acted

Procedural memory is where skills come from.

Agent memory as a service



bi-directional: agents write memories and read back grounded context

Open core

neo4j-agent-memory (Python)

Hosted

NAMS: multi-tenant service

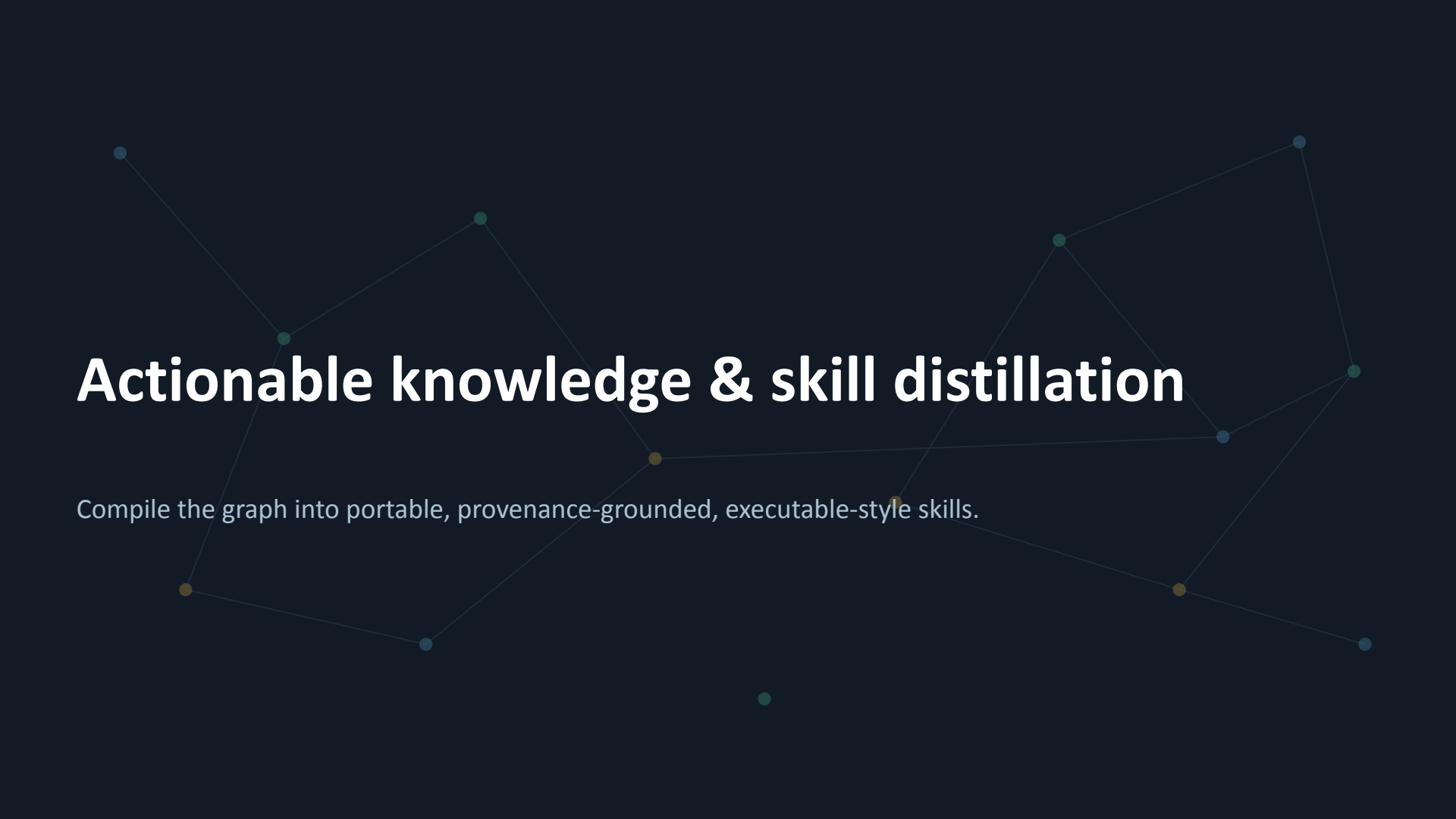
Extraction

LLM entity/relation pipeline

Pure-Go services · Neo4j HTTP Query API · per-workspace isolation · API-key & OAuth/MCP auth

Memory tells you **what happened**.
Skills tell you **what to do**.



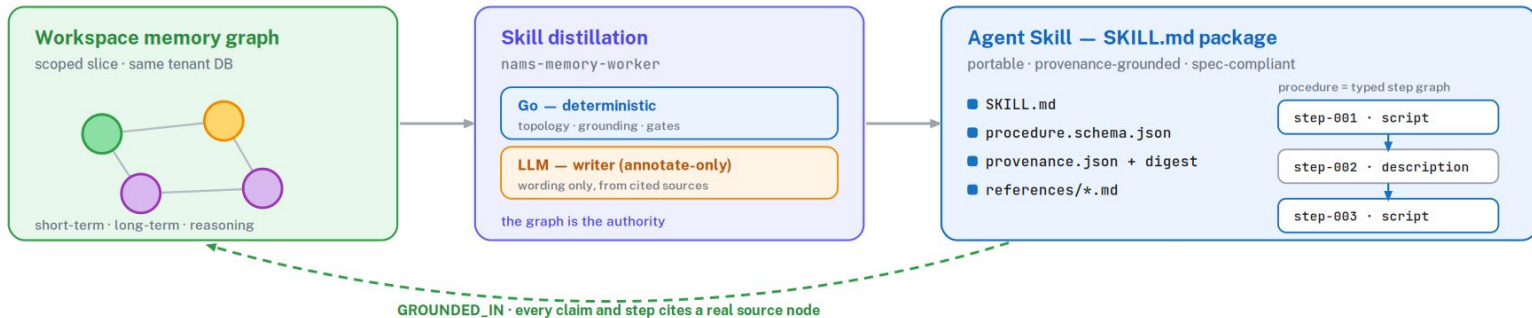
The background features a dark blue-grey field with a network of thin, light-grey lines connecting small, semi-transparent dots. The dots are colored in shades of teal, blue, and brown, creating a complex, web-like pattern that frames the central text.

Actionable knowledge & skill distillation

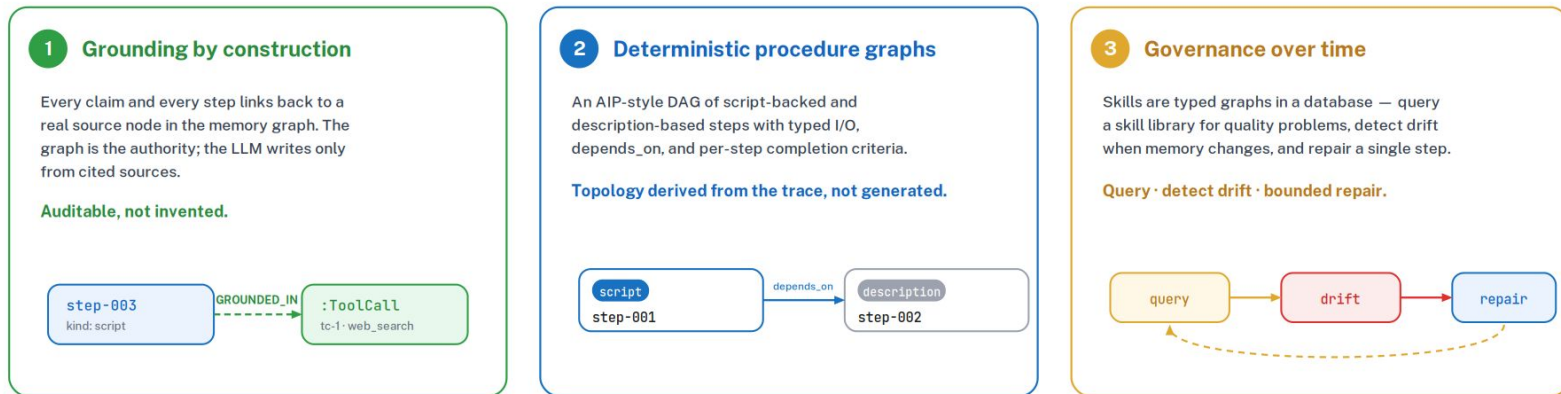
Compile the graph into portable, provenance-grounded, executable-style skills.

Skill distillation: from memory graph to grounded Agent Skill

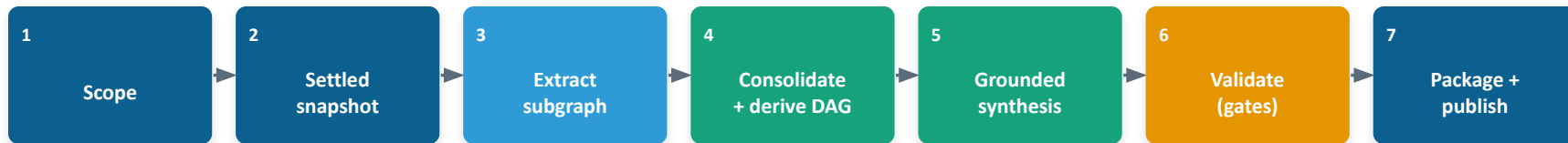
NAMS turns a scoped slice of a workspace's memory graph into a portable, provenance-grounded Agent Skill — procedure as a typed execution graph, not prose.



Three ideas carry the whole design



The distillation pipeline — deterministic-first



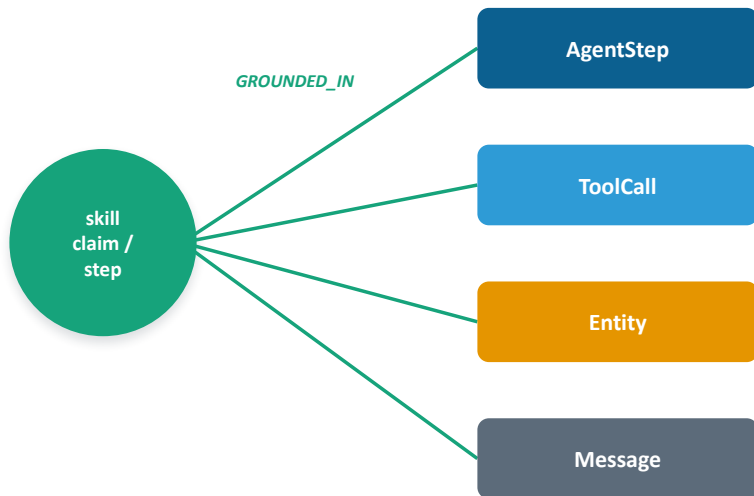
The graph derives the structure

Consolidation clusters recurring decision → action → outcome patterns, weights them by frequency, splits effective vs. anti-pattern by tool-call status, and derives the step DAG — all deterministically from the trace.

The LLM only annotates — from sources

A constrained LLM endpoint may write only from supplied source_refs; the worker re-checks every claim against the live subgraph and drops anything ungrounded. Most systems let the LLM invent structure — exactly where hallucinated procedure creeps in.

Provenance: the graph is the authority



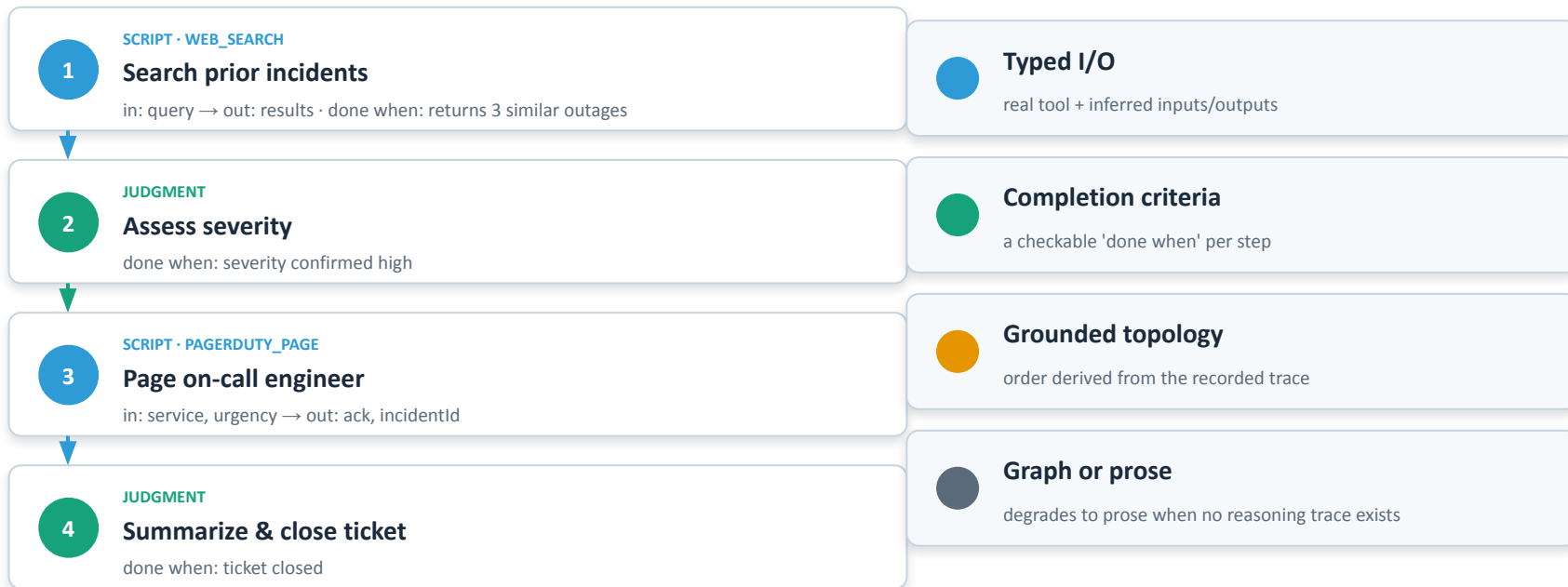
Ungrounded skills (most LLM systems)

The model writes a plausible procedure from a transcript. No link to evidence, no way to verify, no way to tell when it goes stale.

Grounded skills (context graph)

Every claim and step carries edges to the exact memory nodes it came from — re-verified at build time, inspectable at read time.

The AIP procedure graph — typed, executable-style



Same memory, two renderings — measured

procedure-format: graph

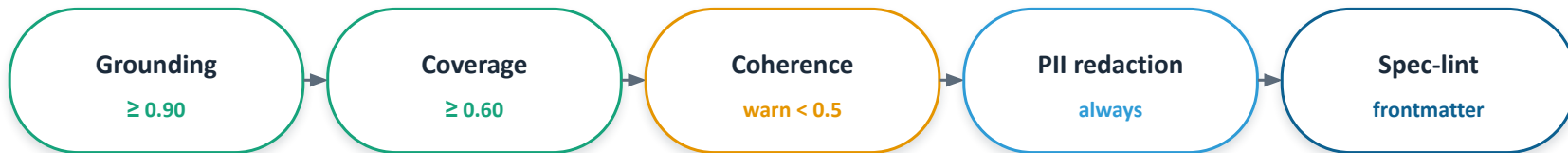
- Typed step DAG inlined in SKILL.md
- A planner can bind tools + check 'done when'
- First-class steps: editable, composable
- Requires reasoning traces in scope

procedure-format: prose

- Body collapses to 'see references/procedures.md'
- Agent must parse prose to act
- No steps to edit, ground, or compose
- The fallback for non-procedural memory

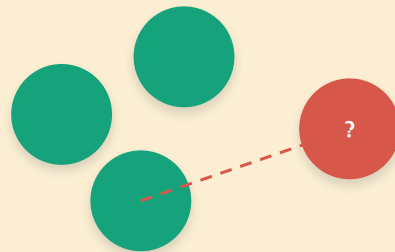
Both arms: 100% grounding. Format isn't a free toggle — it reflects whether the underlying memory is procedural. A live A/B over message-only memory produced prose from both arms; the incident-trace memory produces a graph.

Grounding $\neq$ good: the quality gates



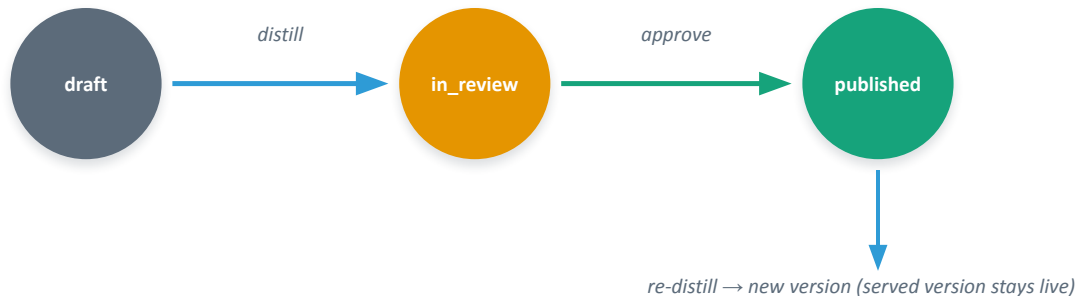
The coherence gate — because 100% grounding can still be a bad skill

An early run distilled the whole workspace and produced a perfectly-grounded skill that chained a billing-dispute step onto incident-management steps. Every claim was real; the skill was incoherent. Grounding proves a skill is real — not that it's one thing.



coherence: now scored, warned, and (optionally) hard-gated with a split suggestion

Governance: review, publish, version



publishedVersionId $\neq$ latest — a stable served version keeps serving while edits pend review.

Corpus governance, computed live:

Reuse candidates

sources shared across skills $\rightarrow$ extract a sub-procedure

Missing success criteria

published skills with no checkable outcome

Drifted

skills whose sources changed

Dynamic loading: discover, verify, load

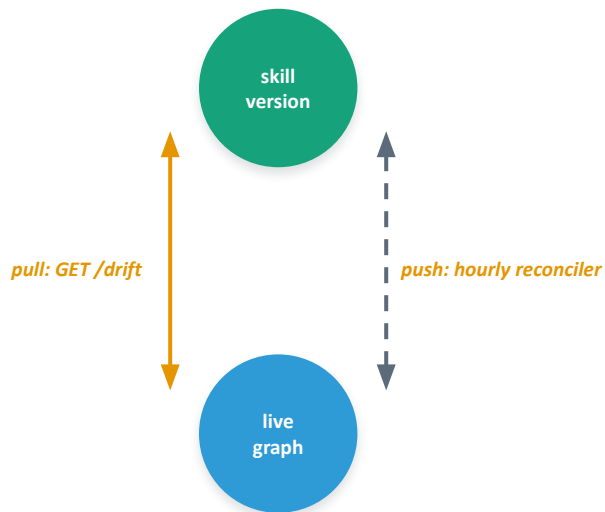


agentskills.io discovery manifest (schema 0.2.0)

```
{ name, description, version,  
  digest: "sha256:...", attested: false,  
  procedureFormat: "graph",  
  downloadUrl, explainUrl }
```

- **Portable** — any agentskills.io-compatible runtime
- **Verifiable** — per-version sha256 digest
- **Signable** — attestation scaffolded (next: turn it on)

Drift: a skill that silently rots is worse than none



Stale

A grounding source was deleted or merged away. The claim lost its evidence.

Contradicted

A recorded tool outcome changed. The procedure may now be wrong, not just unsupported.

SCREENSHOT

Corpus governance:

1 missing success criteria

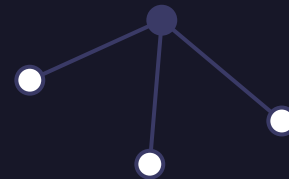
50 reuse candidates

1 drifted

[Details →](#)

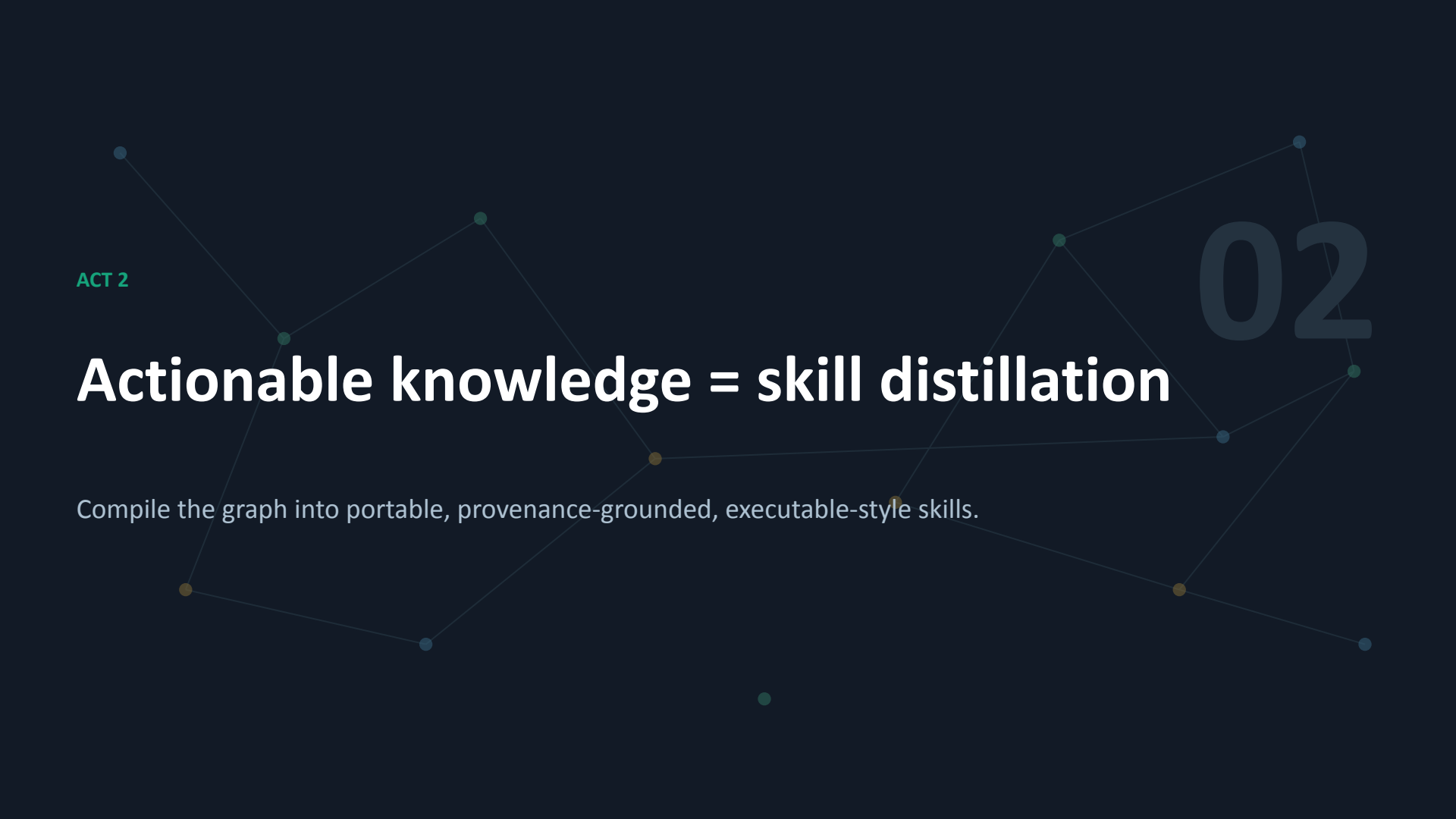
⚠ This skill has drifted from current memory

2 stale claim(s), 0 stale step(s). Repair an individual step below, or re-distill the whole skill.



Skills that run

From prose instructions to runnable steps.



ACT 2

02

Actionable knowledge = skill distillation

Compile the graph into portable, provenance-grounded, executable-style skills.

From prose claims to a typed procedure graph

Same recorded conversation. The graph makes each step addressable, typed, and individually grounded.

Before — prose procedure claims

- Search prior incidents before triaging.
- Assess the severity of the outage.
- Page the on-call engineer for P1 outages.
- Confirm resolution before closing the ticket.

X re-derived from prose each run (slow, token-heavy)

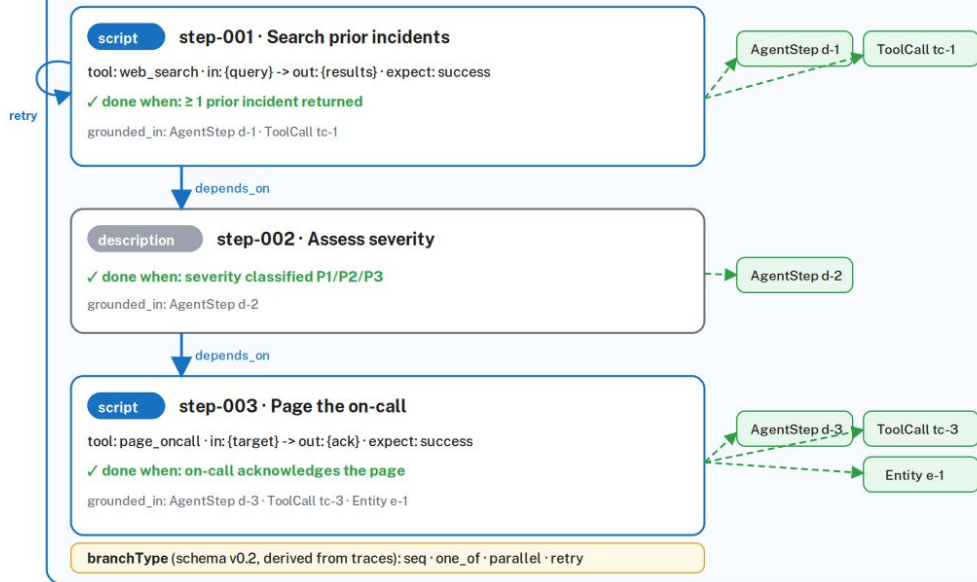
X no addressable units -> opaque failures

X accuracy swings with wording (prompt brittleness)

X "improvement" = unbounded prose edits

Kept for memory-light scopes (< MIN_STEPS): procedureFormat = prose.

After — :SkillStep DAG procedureFormat = graph



Why structure (AIP, SkillsBench)

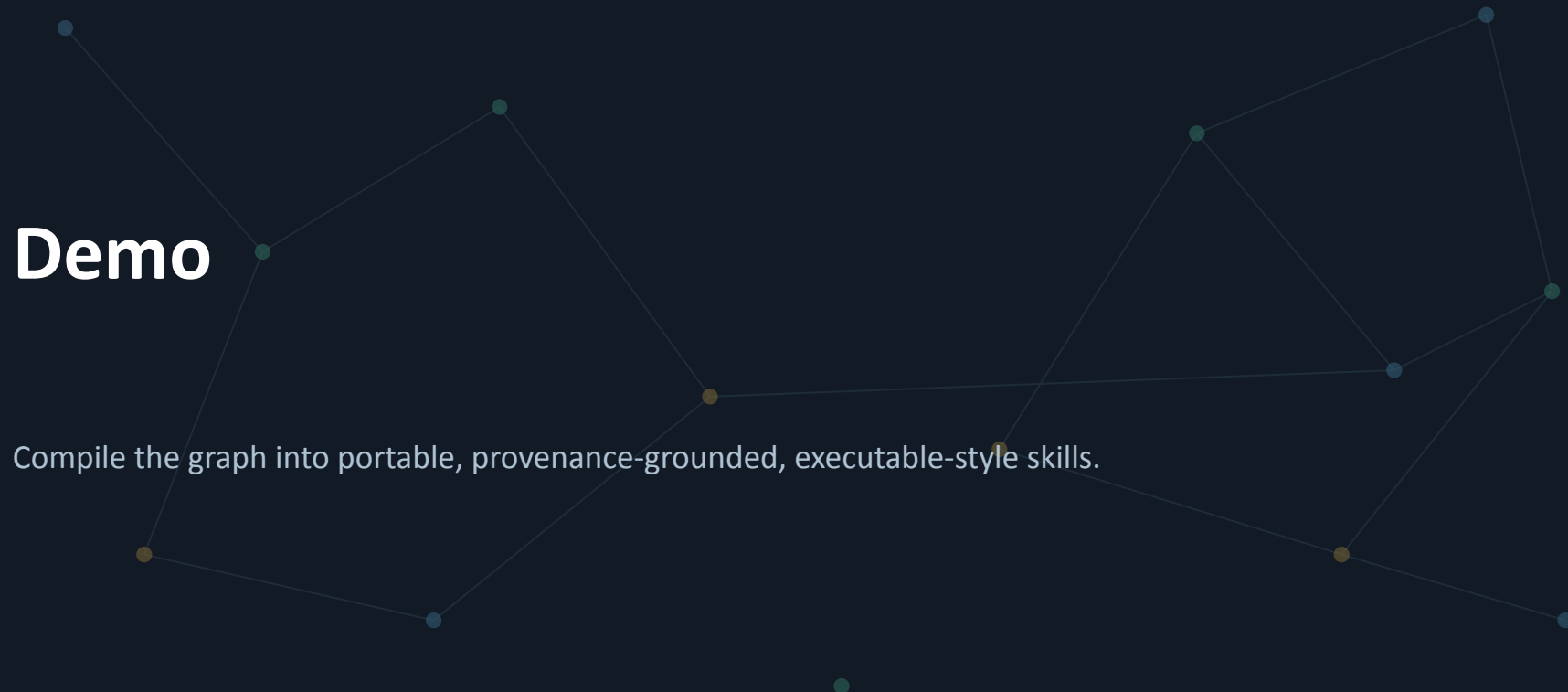
structured beat freeform prose: +14.1pp pass (53.3%→67.4%),
~13% faster (585s→510s), Wilcoxon p = 0.011

arXiv 2606.04781 — verify numbers before publishing

How NAMS keeps it grounded

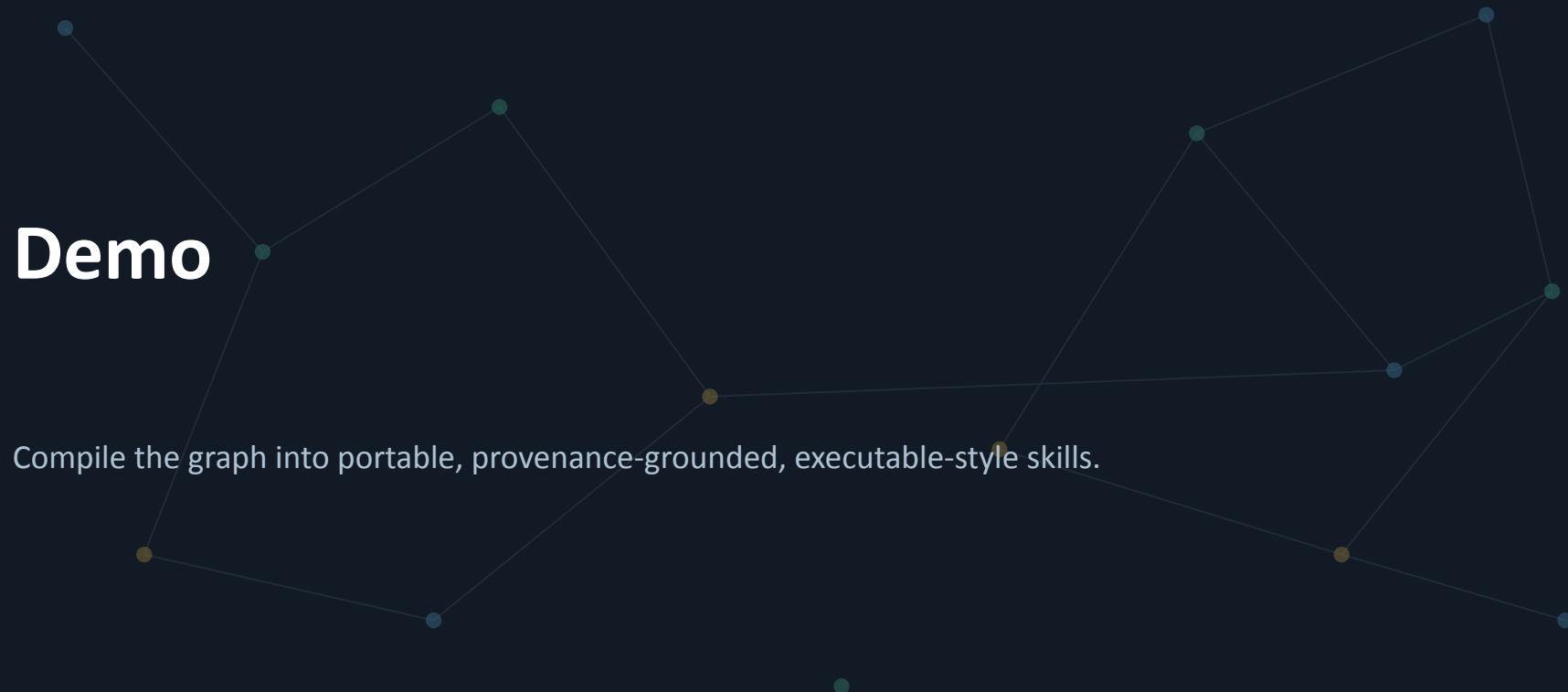
Topology + grounding are derived from the recorded trace
(CreatedAt order; kind=script iff a :ToolCall is present).

The LLM annotates name/description/criterion only — never structure.



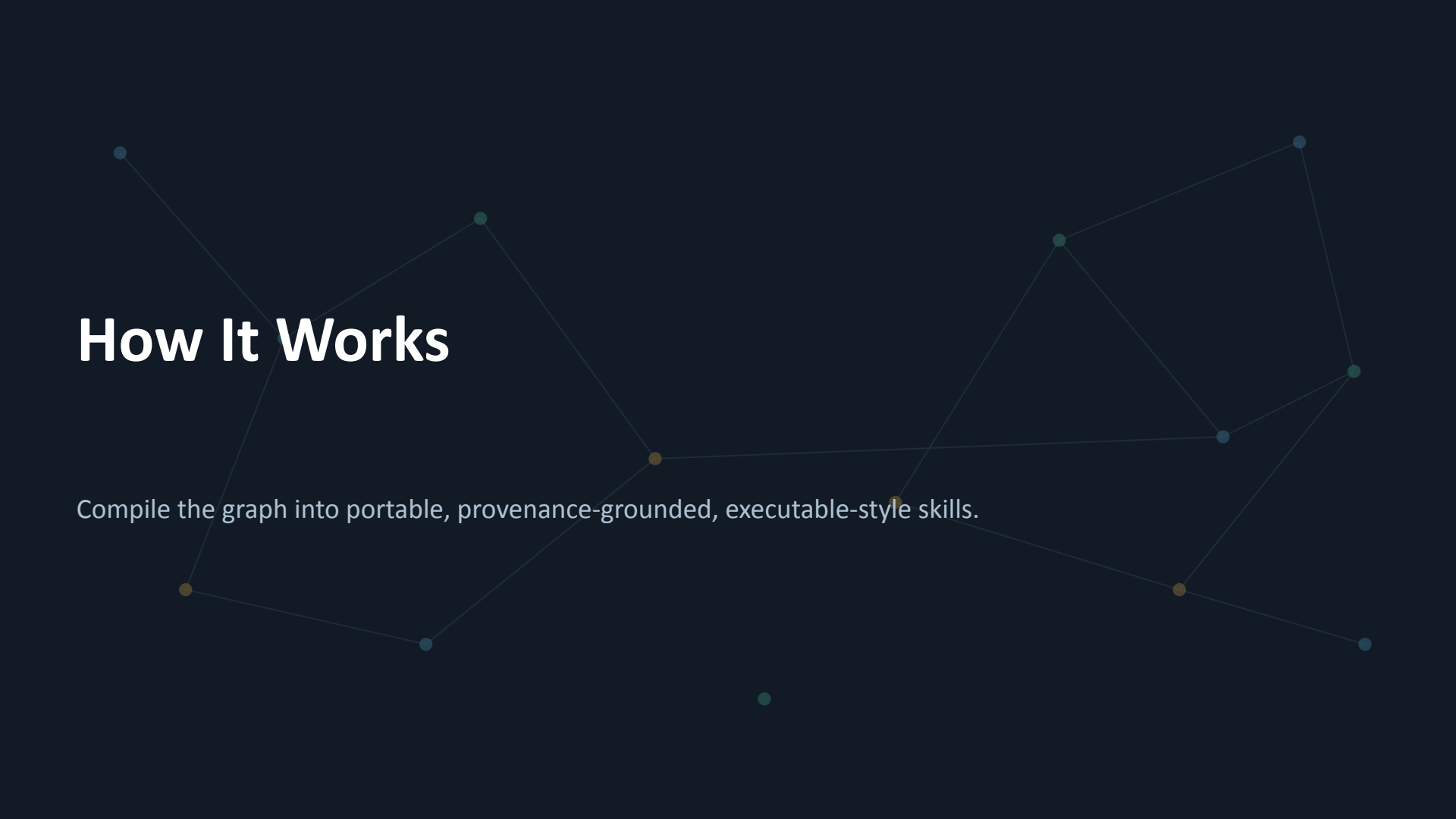
Demo

Compile the graph into portable, provenance-grounded, executable-style skills.



Demo

Compile the graph into portable, provenance-grounded, executable-style skills.

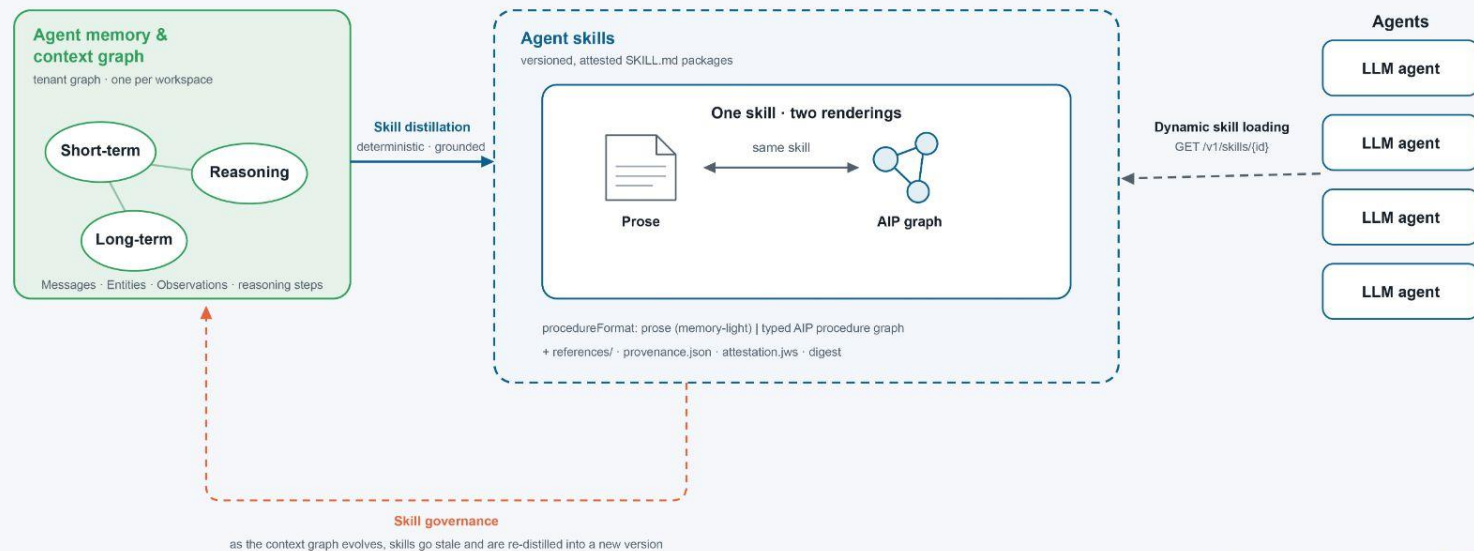
An abstract geometric diagram consisting of approximately 15 points connected by thin, light-colored lines. The points are colored in three distinct shades: a muted blue, a dark teal, and a brownish-gold. The connections form a complex, non-uniform network of polygons and paths across the dark blue background. Some points are isolated, while others are part of larger, more interconnected clusters.

How It Works

Compile the graph into portable, provenance-grounded, executable-style skills.

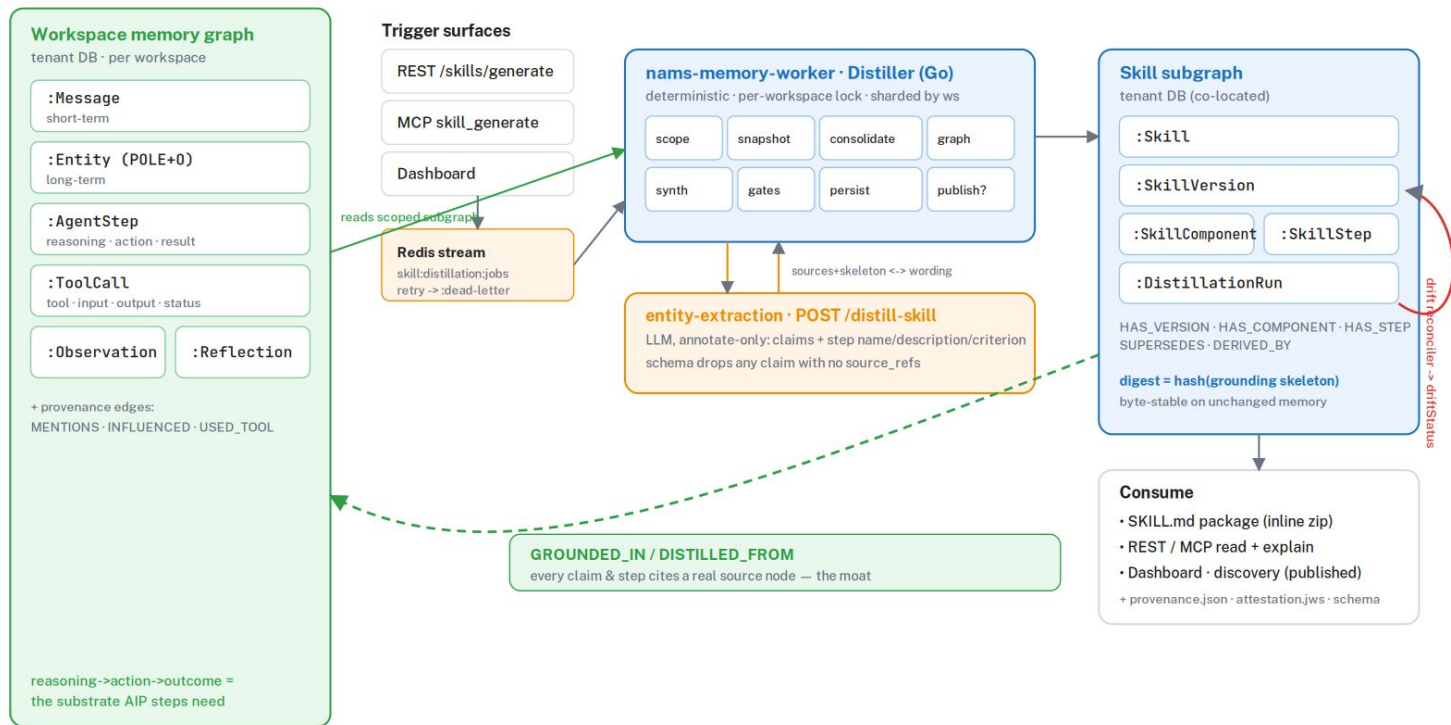
Skill distillation & governance in Neo4j Agent Memory Service (NAMS)

Memory becomes portable skills; agents load them on demand; governance re-distills them as the context graph evolves.



The graph is the authority; the LLM is a writer working strictly from cited sources.

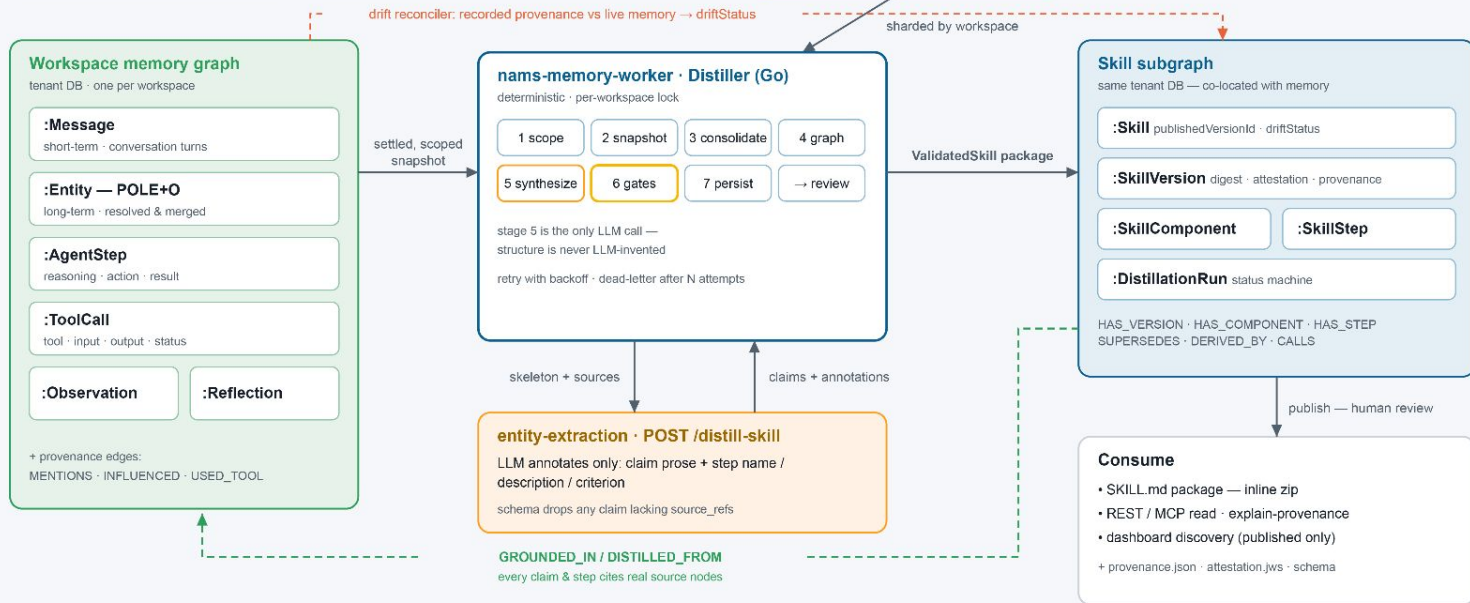
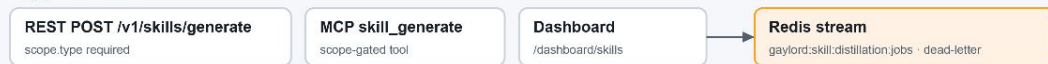
The graph is the authority; the LLM is a writer working strictly from cited sources.



NAMS skill distillation — memory → grounded skill

The graph is the authority; the LLM is a writer working strictly from cited sources.

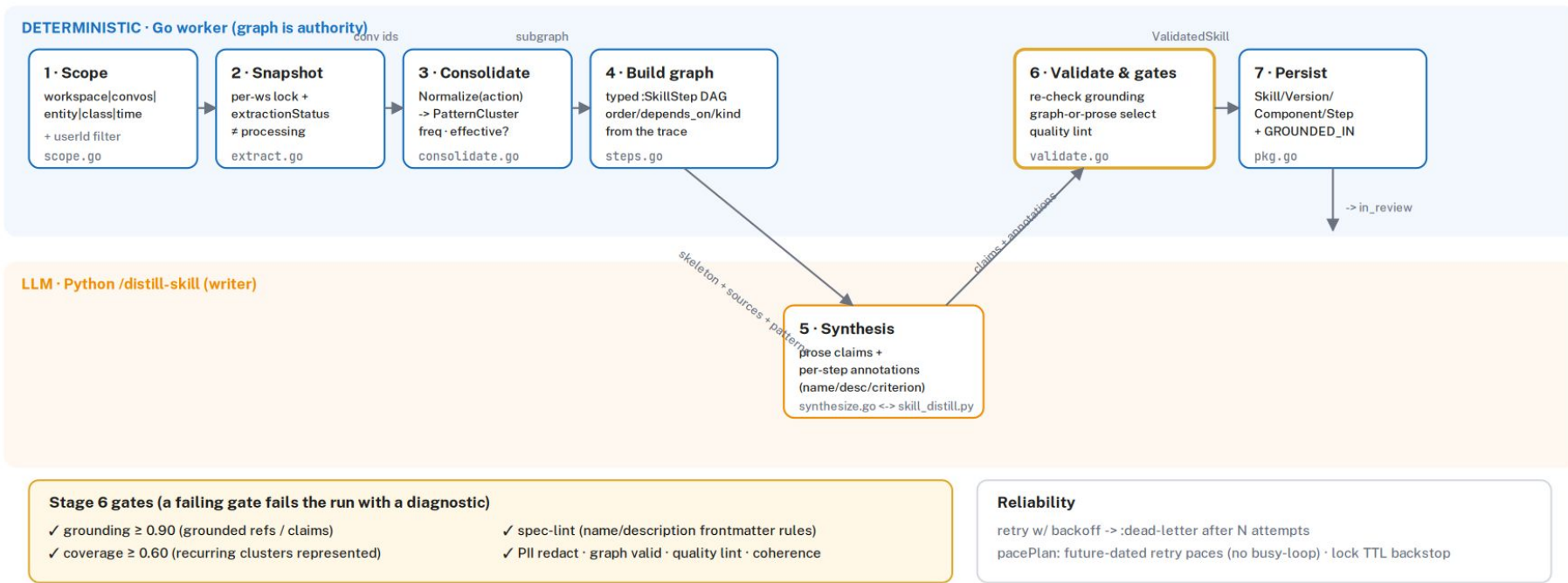
Trigger surfaces



memory (authority) Go (deterministic)
Python (LLM) drift loop

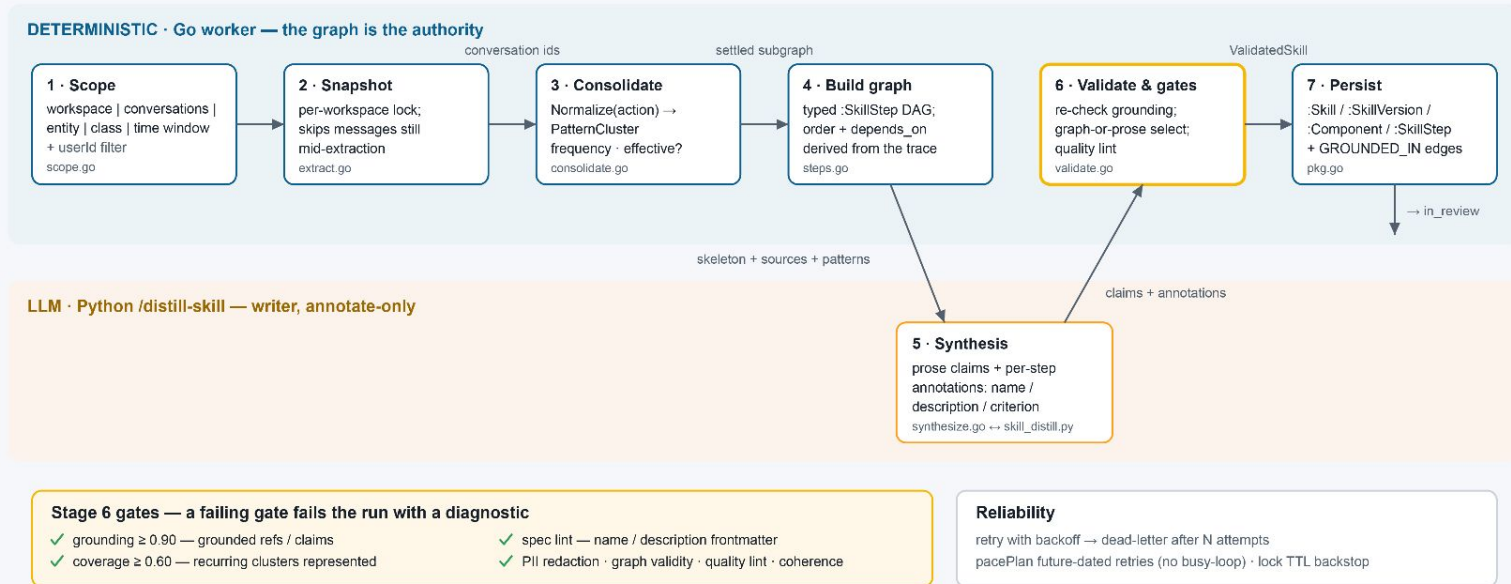
The distillation pipeline — seven stages

Six stages are deterministic Go; only synthesis is LLM, and it annotates wording — it never invents structure.



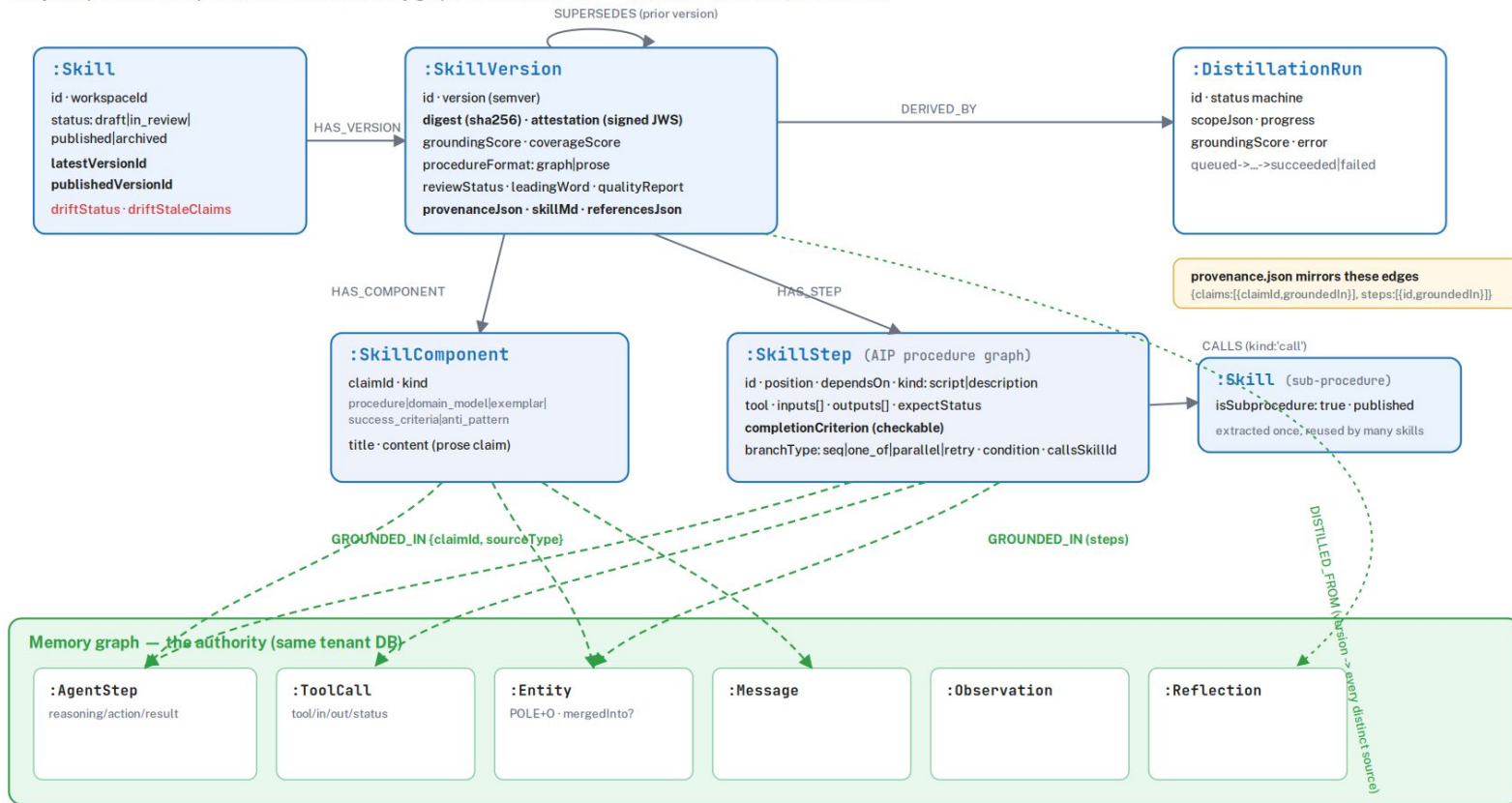
The distillation pipeline — seven stages

Six stages are deterministic Go; only synthesis is LLM, and it annotates wording — it never invents structure.



The skill subgraph and its grounding moat

Every component and step links back into the memory graph via GROUNDED_IN — so a skill is auditable, not invented.

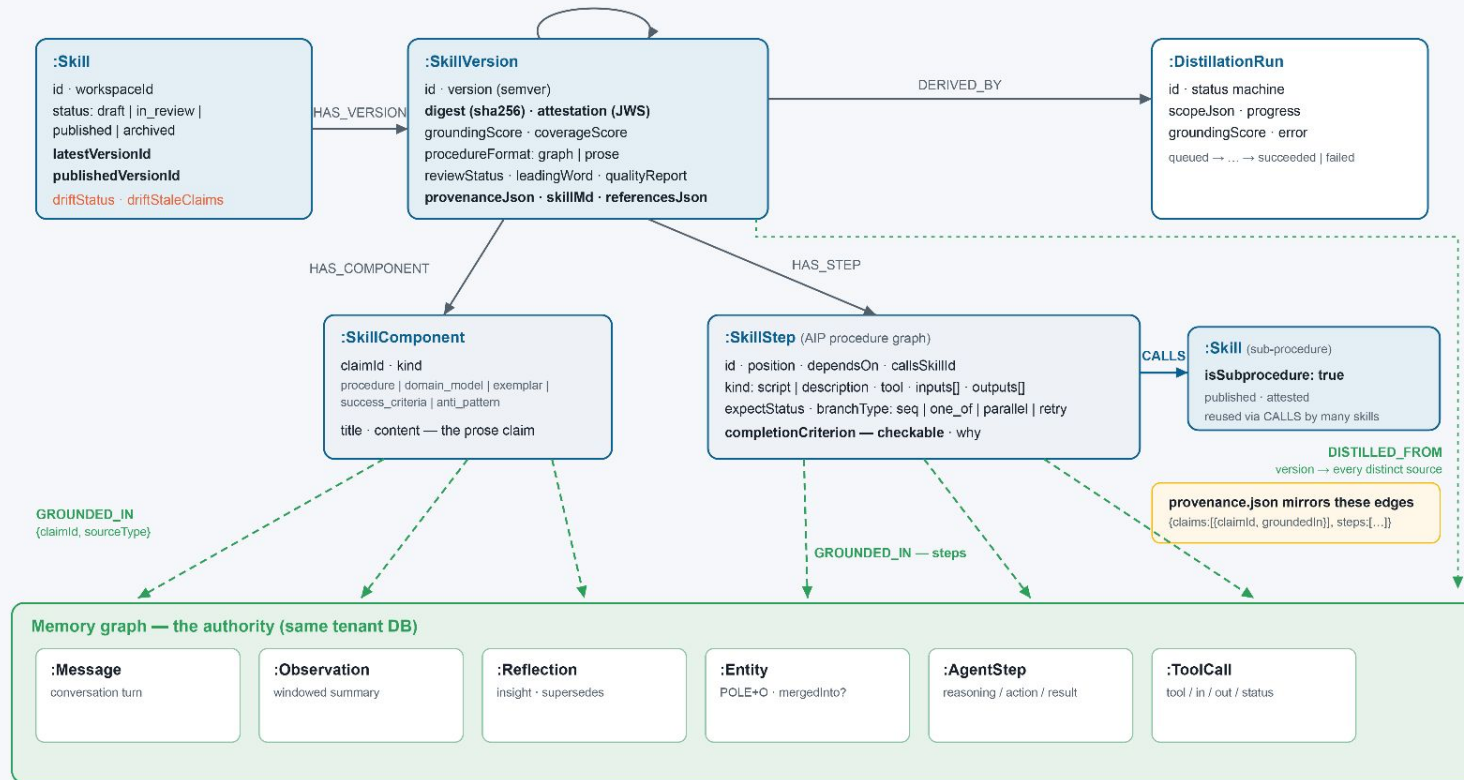


The skill subgraph and its grounding moat

Every component and step links back into the memory graph via GROUNDED_IN — so a skill is auditable, not invented.

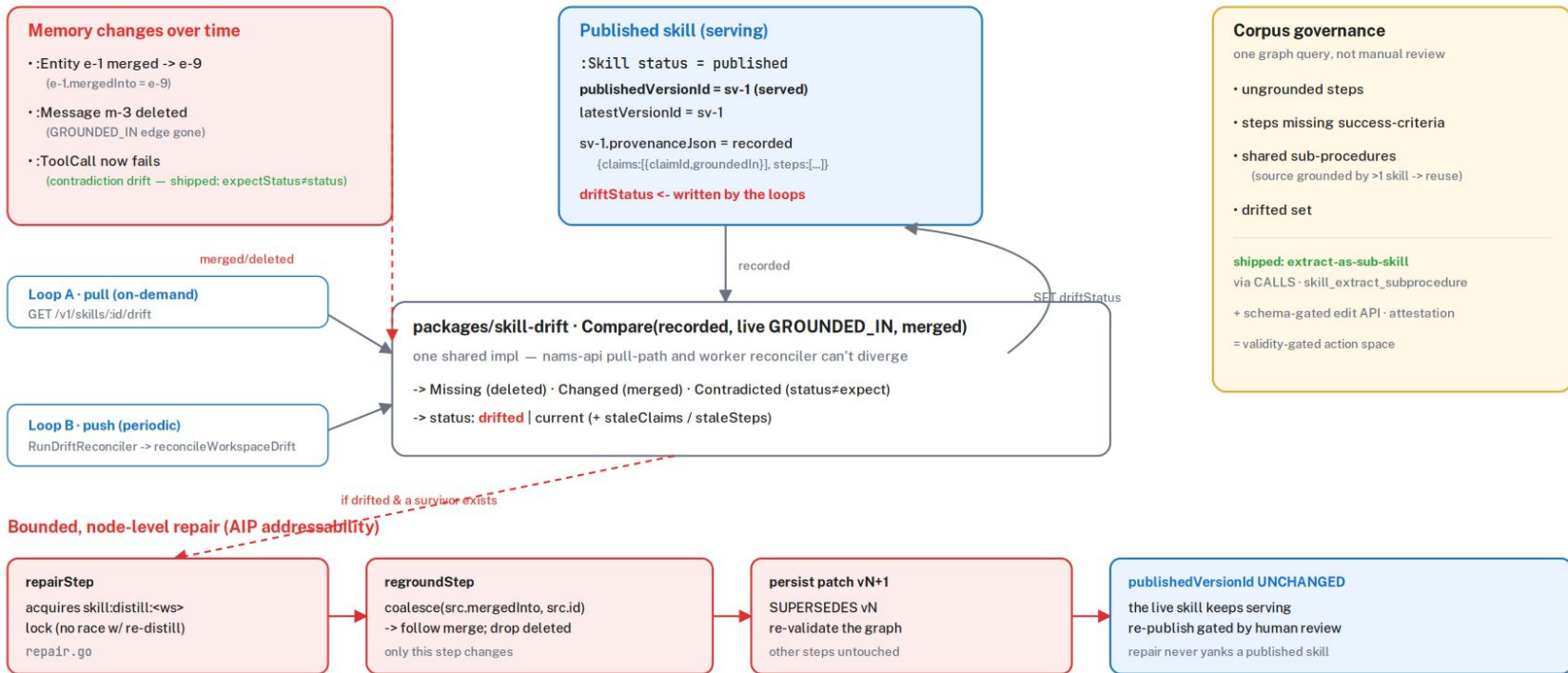
SUPERSEDES → prior version

— structural edge - - - grounding edge



Keeping skills honest: drift double-loop + bounded repair

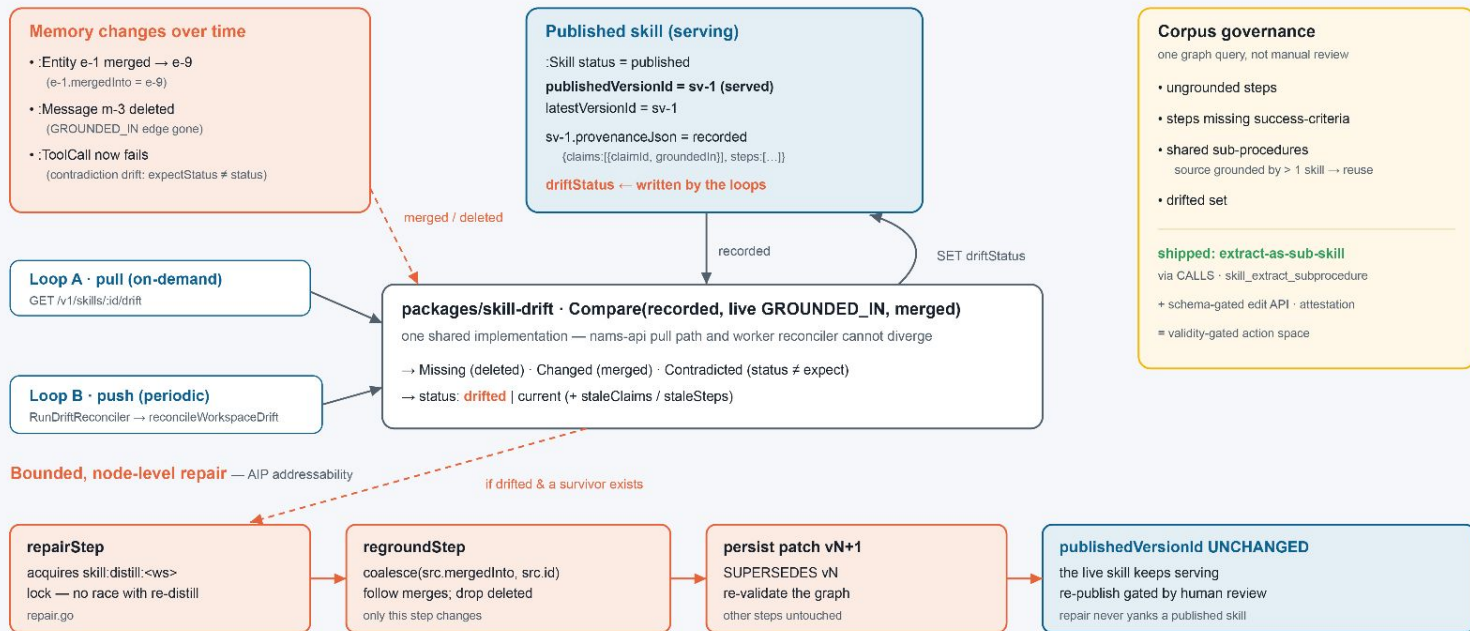
Memory changes; the recorded provenance is re-checked against the live graph; a single drifted step is repaired without un-serving the skill.



If no surviving grounding exists, repair declines and flags a full re-distill — bounded repair fixes salvageable drift, never papers over the unsalvageable.

Keeping skills honest: drift double-loop + bounded repair

Memory changes; recorded provenance is re-checked against the live graph; a single drifted step is repaired without un-serving the skill.



If no surviving grounding exists, repair declines and flags a full re-distill — bounded repair fixes salvageable drift, never papers over the unsalvageable.

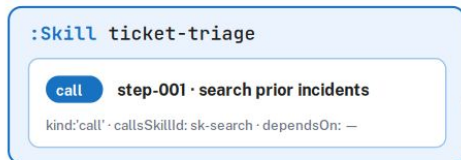
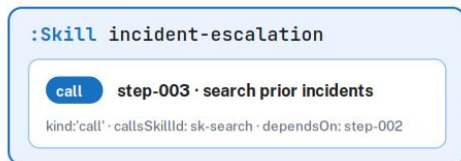
Composition: extract a shared sub-procedure, reference it with CALLS

Governance finds a source grounded by steps of more than one skill; the shared step is promoted to its own reusable :Skill — a single source of truth.

Reuse query (corpus governance) — one source grounded by steps of > 1 skill is a reuse candidate

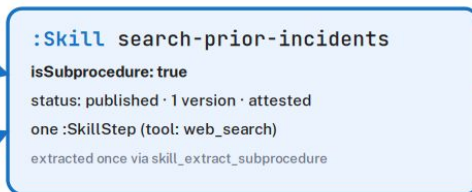
```
MATCH (s:Skill)-[:HAS_VERSION]->(:SkillVersion)-[:HAS_STEP]->(:SkillStep)-[:GROUNDED_IN]->(src) WITH src, collect(DISTINCT s.id) AS skillIds WHERE size(skillIds) > 1 -> skill_extract_subprocedure
```

Caller skills



CALLS

CALLS



GROUNDED_IN

→ CALLS (structural)

--- GROUNDED_IN

Memory graph — the authority

:ToolCall tc-1

web_search · status: success

:AgentStep d-1

reasoning / action / result

Before -> after

Before: both skills carried a duplicate inline step, each grounding straight into :ToolCall tc-1.

After: one extracted :Skill is grounded in tc-1; both callers hold a kind:'call' step that CALLS it — a single source of truth. Topology, grounding, and the digest stay derived from the recorded trace.

Publish gate

A caller cannot publish while a skill it CALLS is unpublished — PublishSkill returns 409 with the offending sub-skill, so the corpus stays consistent.

Composition: extract a shared sub-procedure, reference it with CALLS

→ CALLS (structural)

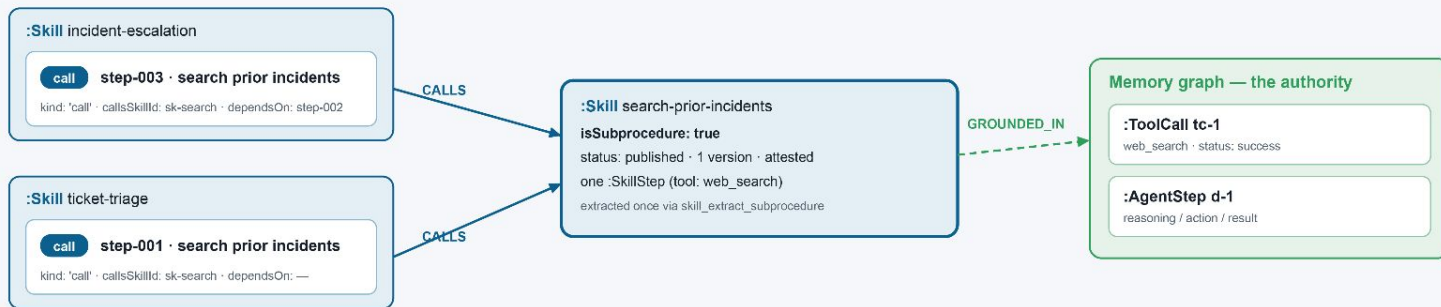
--- GROUNDED_IN

Governance finds a source grounded by steps of more than one skill; the shared step is promoted to its own reusable :Skill — a single source of truth.

Reuse query (corpus governance) — one source grounded by steps of > 1 skill is a reuse candidate

```
MATCH (s:Skill)-[:HAS_VERSION]->(:SkillVersion)-[:HAS_STEP]->(:SkillStep)-[:GROUNDED_IN]->(src) WITH src, collect(DISTINCT s.id) AS skillIds WHERE size(skillIds) > 1
```

Caller skills



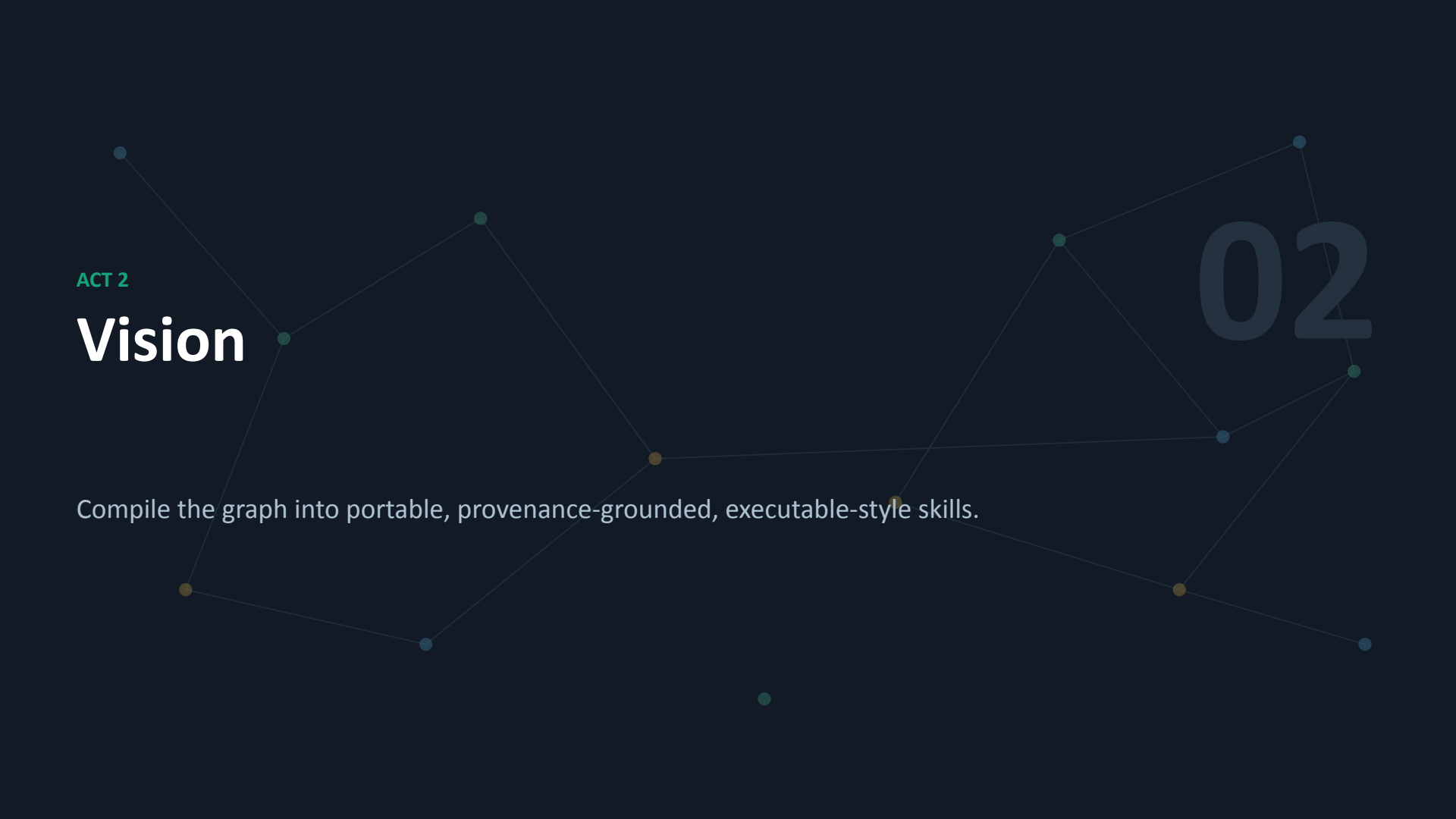
Before → after

Before: both skills carried a duplicate inline step, each grounding straight into :ToolCall tc-1.

After: one extracted :Skill is grounded in tc-1; both callers hold a kind: 'call' step that CALLS it — a single source of truth. Topology, grounding, and the digest stay derived from the recorded trace.

Publish gate

A caller cannot publish while a skill it CALLS is unpublished — PublishSkill returns 409 with the offending sub-skill, so the corpus stays consistent.



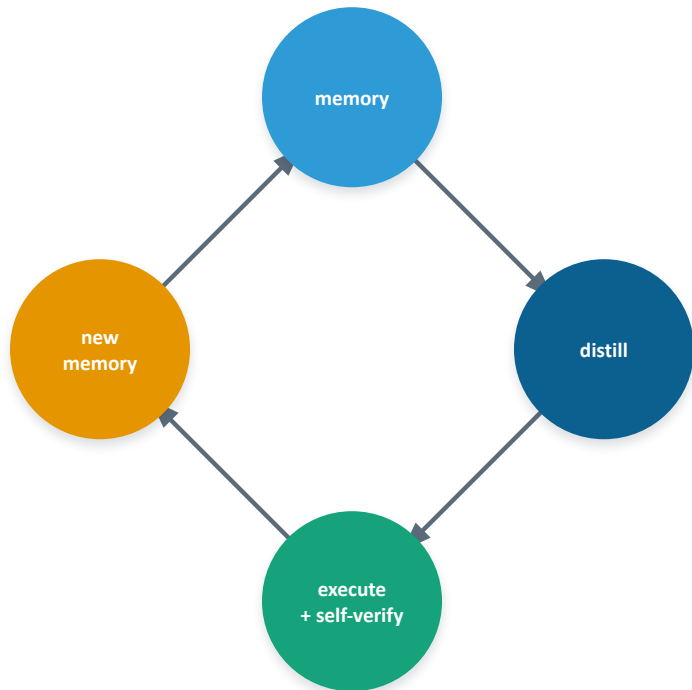
ACT 2

Vision

02

Compile the graph into portable, provenance-grounded, executable-style skills.

Documents → capabilities



Today the procedure graph is a grounded spec. Bind each script step to a live tool, run it, and check the completion criterion at runtime — Voyager's self-verification, over a grounded postcondition.

R1 Executable + self-verifying skills

execution writes new memory → the next distillation is better

R2 Incremental delta curation

patch claims/steps, preserve grounded detail (no context collapse)

Now · Next · Frontier

NOW

- Executable, self-verifying procedure graph (R1)
- Incremental delta updates — the ACE curator (R2)

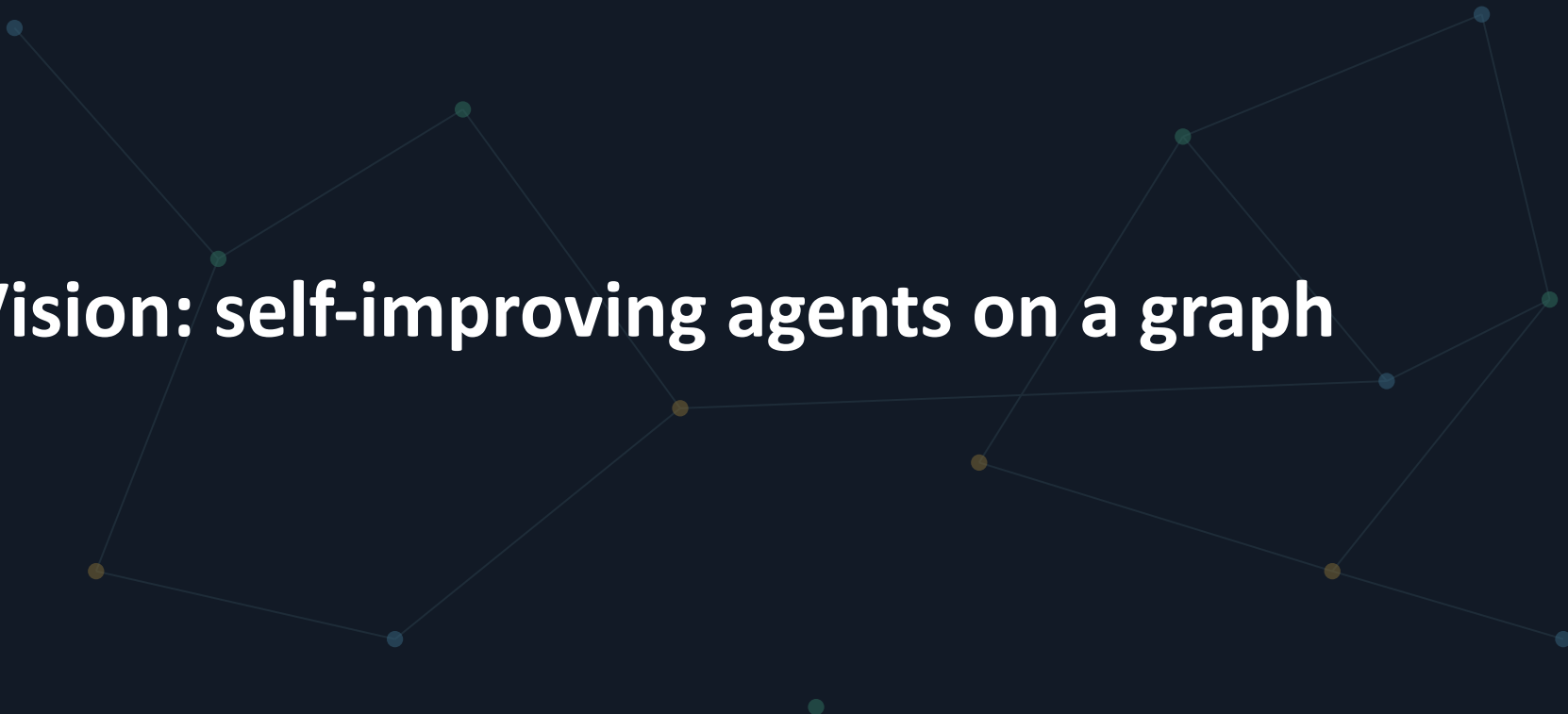
NEXT

- MDL compression → auto sub-procedures (LILO)
- Pre/postconditions + real branching
- Held-out pass-rate eval gates publication

FRONTIER

- Signed skills → transfer-eval-ranked marketplace
- Self-authoring agents, gated by governance
- Provenance-tiered serving (escalate to evidence)

Import the field's mature machinery — executability, compression, delta-curation, real eval — onto the provenance backbone none of them has.

An abstract graph structure is visible in the background, consisting of several nodes (colored blue, green, and brown) connected by thin, light blue lines. The nodes are scattered across the frame, with some forming clusters and others isolated. The overall shape of the graph is irregular and spans most of the width of the image.

Vision: self-improving agents on a graph

The background is a dark navy blue. It features a network of thin, light blue lines connecting various nodes. The nodes are small circles in shades of blue, green, and brown, scattered across the frame. Some nodes are connected by lines, forming a web-like structure, while others are isolated.

THE VISION

Context graphs are the substrate for self-improving agents.

Actionable, grounded, governed knowledge — that an agent can discover, trust, run, and improve.

Resources

graphstuff.com/actionknowledge



GraphAcademy

graphacademy.neo4j.com

Free courses on Neo4j, Graph Data Science, and GenAI

neo4j-agent-memory

github.com/neo4j-labs/agent-memory

Graph-native memory system for AI agents

create-context-graph

create-context-graph.dev

Scaffold full-stack context graph apps in seconds

Neo4j Agent Memory Service (NAMS)

memory.neo4j.com

Agent memory as a service

Documentation

neo4j.com/labs/agent-memory

Tutorials, how-to guides, API reference, concepts

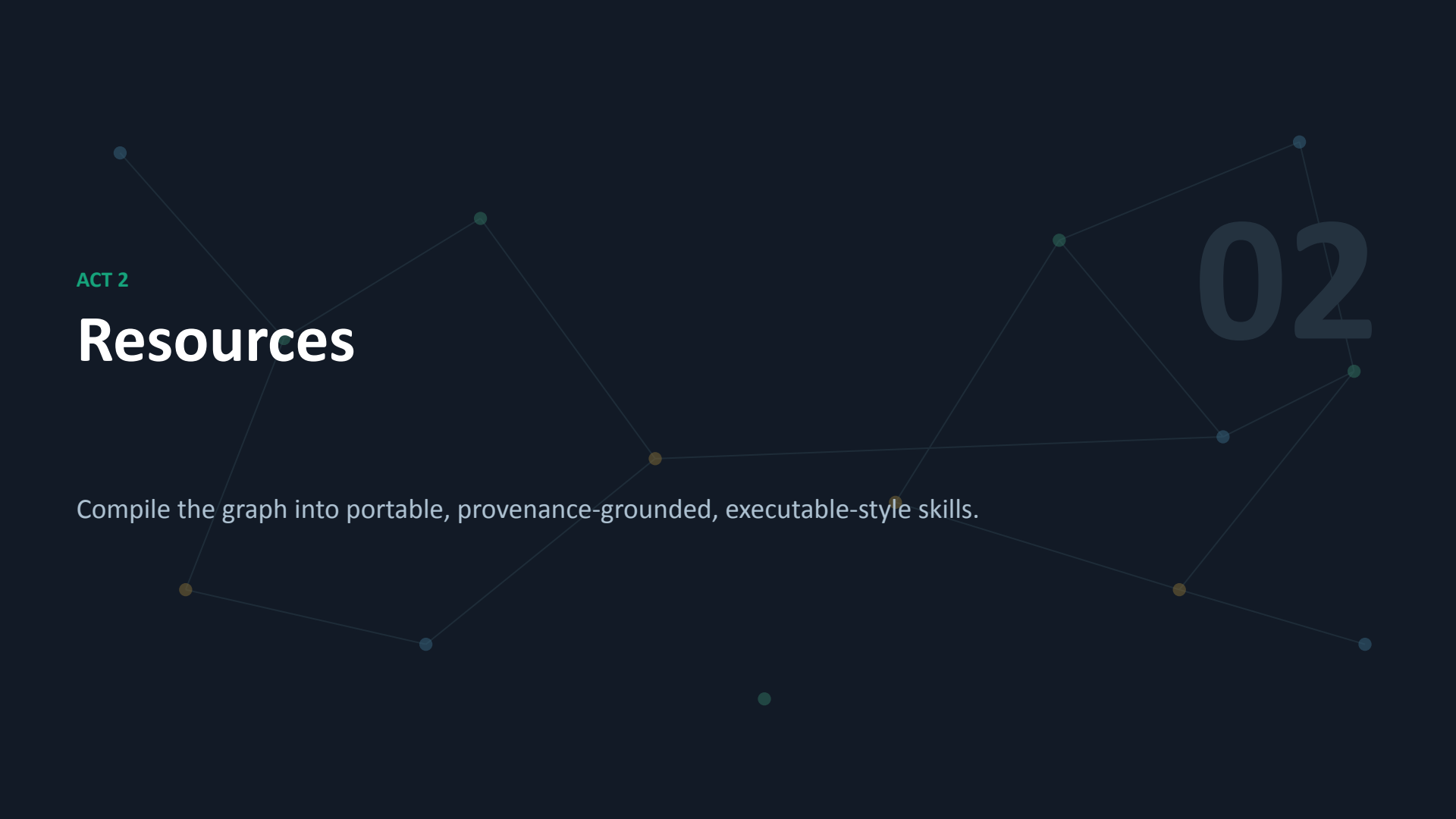
Community Forum

community.neo4j.com

Ask questions, share feedback, connect with the team

Thank you!

William Lyon • @lyonwj • github.com/johnnymontana



ACT 2

Resources

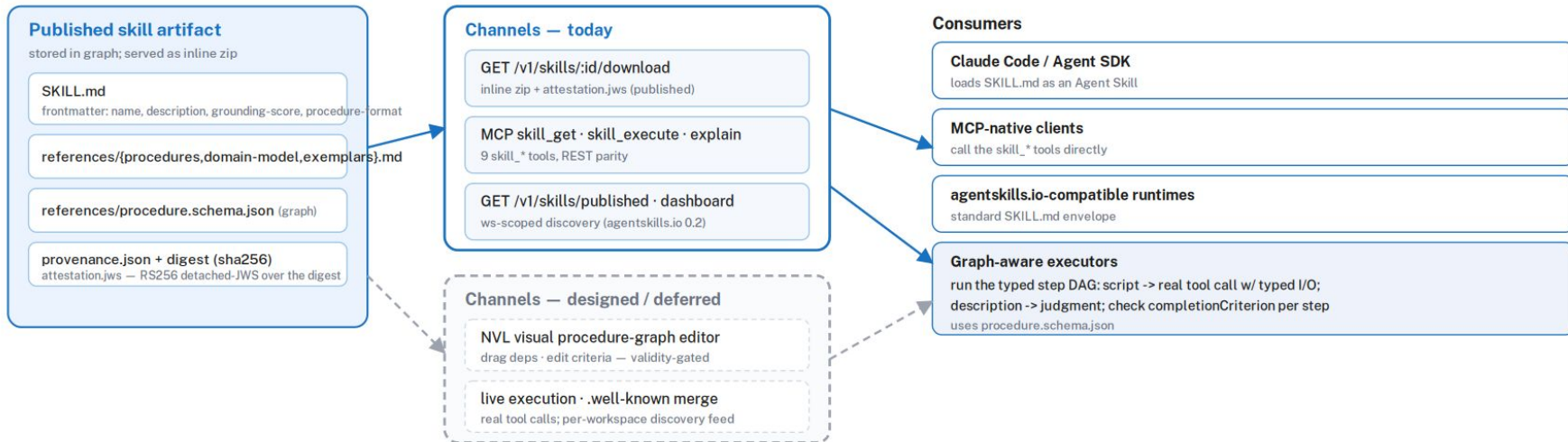
02

Compile the graph into portable, provenance-grounded, executable-style skills.

Distribution & framework integration

A spec-compliant SKILL.md + bundled procedure.schema.json loads anywhere; solid = shipped, dashed = designed/deferred.

— shipped — — — deferred



The prize: a validity-gated action space for self-improving skills

node addressability + schema gate make skill improvement a structured, testable loop — not fragile prose-rewriting

1 · diagnose

governance query / drift
flags a step or gap

2 · propose edit

POST /skills/id/edit (dry-run valid):
reorder/retype/reground · extract sub-skill

3 · human-gated publish

review -> published;
SUPERSEDES lineage

4 · evaluate

graph-vs-prose transfer eval
(scripts/skill_transfer_eval.py) -> pass-rate

measured reward feeds the next diagnosis